Atlas AI

Offline Multi-Expert AI System with Edge Device Computation

Major Project

Project report submitted in partial fulfilment of the requirement for the award of the Degree

Bachelor of Technology (B. Tech)

in

COMPUTER SCIENCE AND ENGINEERING

by

A. Madhav	**22AG1A0503**
V. Tarun	**22AG1A0555**
B. Pramod	**22AG1A0507**

Under the Esteemed

Guidance of

Mr. A. Ramesh

Associate Professor & HoS

Department of Computer Science and Engineering

ACE ENGINEERING COLLEGE

An AUTONOMOUS Institution

NBA Accredited B. Tech Courses, Accorded NAAC 'A' Grade

(Affiliated to Jawaharlal Nehru Technological University,Hyderabad,Telangana)

Ankushapur (V), Ghatkesar (M), Medchal – Malkajgiri Dist - 501 301.

April – 2026

ACE

Engineering College

An Autonomous Institution

NBA Accredited B. Tech Courses, Accorded NAAC 'A' Grade

Website: www.aceec.ac.in E-mail: info@aceec.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Major project work entitled **"Atlas AI – Offline Multi-Expert AI System with Edge Device Computation"** is being submitted by **A. MADHAV (22AG1A0503), V. TARUN (22AG1A0555), B. PRAMOD (22AG1A0507)** in partial fulfillment of the award of Degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING** to the Jawaharlal Nehru Technological University, Hyderabad during the academic year 2025-2026 is a record of Bonafide work carried out by them under our guidance and supervision. The results embodied in this report have not been submitted by the student to any other University or Institution for the award of any degree or diploma.

Internal Guide	HoS	HOD – CSE
Mr. A. Ramesh	**Dr. Ch. Vijaya Kumar**	**Dr. Ch. Vijaya Kumar**
Associate Professor & HOS	**Professor & HOD - CSE**	**Professor & HOD - CSE**

Project Coordinator	EXTERNAL EXAMINER
Ms. Shubhangi Mahule	
Associate Professor	

ACKNOWLEDGEMENT

We would like to express our gratitude to all the people behind the screen who have helped us transform an idea into a real time application.

We would like to express our heart-felt gratitude to our parents without whom we would not have been privileged to achieve and fulfill our dreams.

A special thanks to our General Secretary, **Prof. Y. V. Gopala Krishna Murthy,** for having founded such an esteemed institution. Sincere thanks to our COO **Mr. Y. V. Raghu Vamshi**, for his support in doing project work. We extend our sincere thanks to **Mrs. Mamatha Rao P**, Director, HR, Administration & Operations for her continuous and dedicated efforts in providing all the necessary facilities for the successful completion of the project. We are also grateful to our beloved Principal, **Dr. N. Sudhakar Reddy** for permitting us to carry out this project. We are very much thankful to our beloved Vice Principal & Dean-Skill Development **Dr. M. Murali** for his continuous encouragement to fulfill our project.

We profoundly thank **Dr. Ch. Vijaya Kumar**, Professor and Head of Department of Computer Science and Engineering, who has been an excellent guide and also a great source of inspiration for our work.

We extremely thank **Dr. Ch. Vijaya Kumar**, HoS & Professor, who helped us in fulfillment of all the aspects for completion of our Major-Project.

We sincerely thank **Ms. Shubhangi Mahule**, **Associate Professor**, for her continuous support and guidance throughout the project.

We are very thankful to our internal guide **Mr. A. Ramesh**, **Associate Professor**, who has been an excellent support for the Completion of our project work.

The satisfaction and euphoria that accompany the successful completion of the task would be great, but incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success. In this context, we would like to thank all the other staff members, both teaching and non-teaching, who have extended their timely help and eased our task.

V. TARUN (22AG1A0555)

A. MADHAV (22AG1A0503)

B. PRAMOD (22AG1A0507)

DECLARATION

We hereby declare that this major project entitled "**Atlas AI – Offline Multi-Expert AI System with Edge Device Computation**" submitted to the ACE Engineering College, is a record of an original work done by us under the guidance of **Mr. A. Ramesh**, Associate Professor & HoS, **ACE Engineering College**, and this project work submitted in the partial fulfillment of the requirements for the major project; the results embodied in this thesis have not been submitted to any other university or institute for award of any degree or diploma.

V. TARUN	(22AG1A0555)
A. MADHAV	(22AG1A0503)
B. PRAMOD	(22AG1A0507)

INDEX

CONTEXT	PAGE-NO
ABSTRACT	
CHAPTER-1: INTRODUCTION	
1.1 MOTIVATION	9
1.2 INTRODUCTION	9
1.3 PROBLEM STATEMENT	10
1.4 OBJECTIVE OF THE PROJECT	10
1.5 SCOPE OF THE PROJECT	10
CHAPTER-2: LITERATURE SURVEY	12 - 13
CHAPTER-3: SOFTWARE REQUIREMENT ANALYSIS	
3.1 PROBLEM STATEMENT	14
3.2 PROPOSED SYSTEM	14
CHAPTER-4: SOFTWARE DESIGN	
4.1 DATA FLOW DIAGRAMS	15 - 17
4.2 UML DIAGRAMS	18 - 28
CHAPTER-5: SOFTWARE & HARDWARE REQUIREMENTS	29
CHAPTER-6: MODULES	
6.1 MOBILE CLIENT MODULE	30
6.2 API SERVER MODULE	30
6.3 AI PROCESSING MODULE	31
6.4 DATABASE MODULE	31
6.5 DESKTOP CORE MODULE	31
CHAPTER-7: IMPLEMENTATION	32

CHAPTER-8: TESTING 32

CHAPTER-9: OUTPUT SCREENS 45 – 47

CHAPTER-10: DEPLOYMENT OF THE PROJECT 8 – 51

CHAPTER-11: INTEGRATION & EXPERIMENTAL RESULTS 52 54

CHAPTER-12: PERFORMANCE EVALUATION 55 – 57

CHAPTER-13: COMPARISON WITH EXISTING SYSTEM 58 – 59

CHAPTER-14: CONCLUSIONS 60

CHAPTER-15: FURTHER ENHANCEMENTS 61 – 63

CHAPTER-16: REFERENCES 64 – 65

ABSTRACT

Atlas AI is a system designed to offer privately controllable offline-first conversational assistant. Atlas AI aims to provide a platform for conversational interactions that can be conducted off the cloud without any dependency upon internet connectivity. In the wake of rising needs for conversational AI interactions that offer enhanced privacy levels, less reliance on third parties, and greater accessibility irrespective of connectivity issues, Atlas AI offers itself as a viable solution to these problems. It seeks to be an offline-first application that emphasizes control by the users over their interactions.

Atlas AI is designed for conversational interaction supported by retrieval-augmented generation (RAG), where relevant information is retrieved from domain-specific local knowledge sources and incorporated into the response generation process. It allows the system to produce responses that are more context-aware and better aligned with the intended domain, while still preserving offline functionality. This application would be built with the purpose of modularity and flexibility in mind; thus, apart from being a largely on-device application, there would also exist an option to extend the application over a local network like desktop or server, if necessary. This means that there will be no use of cloud-based inferencing, analytics, or accounts.

The importance of Atlas AI is in the effort to steer the development of conversational AI towards a self-reliant, privacy-focused and implementation-friendly architecture. The incorporation of offline-first behavior and response production using retrieved answers is meant to ensure relevance and access without relying on third-party systems for performing the core interactions. This way, Atlas AI is an approach where privacy and autonomy are the driving factors, as opposed to simply presenting a technological solution.

LIST OF FIGURES

Figure No.	Exact Figure Title	Page No.
4.1.1	DFD Level 0	15
4.1.2	DFD Level 1	16
4.2.1	Use Case Diagram	17
4.2.2	Class Diagram	18
4.2.3	Sequence Diagram	19
4.2.4	Collaboration Diagram	20
4.2.5	Deployment Diagram	21
4.2.6	Activity Diagram	22
4.2.7a	Component Diagram	23
4.2.7b	Component Diagram	23
4.2.8	State Chart Diagram	25
4.2.9	Object Diagram	26
4.3.1	E-R Diagram	27
6.1.1	MOBILE CLIENT MODULE	30
6.2.1	API SERVER MODULE	32
6.3.1	AI Processing Module	34
6.4.1	Database Module (SQLite)	36

Figure No.	Exact Figure Title	Page No.
6.5.1	DESKTOP CORE MODULE	38
9.1.1	Welcome Screen	49
9.2.1	Chat History / Side Menu	50
9.3.1	Manage Models Screen	51
9.4.1	Settings Screen	52
9.5.1	Chat Interface Screen	53
9.5.2	Chat Interface Screen	53
13.1	Response latency comparison	63
13.2	Token generation throughput	64
13.3	Privacy & offline capability radar	65
13.4	Memory utilization during inference	66
13.5	Feature comparison overview	67

LIST OF ABBREVIATIONS

S. No.	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	NLP	Natural Language Processing
4	STT	Speech-to-Text
5	ASR	Automatic Speech Recognition
6	API	Application Programming Interface
7	UI	User Interface
8	DB	Database
9	JSON	JavaScript Object Notation
10	RAG	Retrieval-Augmented Generation
11	LLM	Large Language Model
12	GPT	Generative Pre-trained Transformer
13	GGUF	GPT-Generated Unified Format
14	SSE	Server-Sent Events
15	CPU	Central Processing Unit
16	APK	Android Package Kit
17	RN	React Native

LIST OF TABLES

Table No.	Table Title	Page No.
2.1	Literature Survey	10
7.2.4.1	Conversations	42
7.2.4.2	Messages	43
7.2.4.3	Settings	43
8.2.1	Black box Testing	46
8.3.1	White box testing	47
8.4.1	Test Case 1 - Chat Functionality	47
8.4.2	Test Case 2: Model Download	48
8.4.3	Test Case 3: Mobile Connection	48
12.2.1	Performance Summary	62

CHAPTER 1
INTRODUCTION

1.1 Motivation

With the growing rate at which Artificial Intelligence (AI) is evolving, there has been a rise in the use of AI-based assistants in numerous daily activities. Unfortunately, the vast majority of AI algorithms utilize cloud computing technology, which imposes some significant constraints on their performance, including privacy concerns, dependence on internet connectivity, latency problems, and lack of user control over data.

Moreover, considering the heightened importance of privacy in the digital age, there is a pressing need for technologies that offer full ownership and control over the users' data. Therefore, the primary motivation here becomes to create an AI-based system that would run completely on the user's device while transferring none of its data to the outside world.

In order to solve these problems, this project is motivated to build Atlas AI, a completely offline privacy-focused AI chatbot that does not require any connection with cloud-based services. It will allow the user to do all computations and operations on their device while maintaining privacy at all times.

1.2 Introduction

Over the last decade, there have been many advances in the field of AI, with almost all software relying on AI. However, such developments come with risks such as privacy issues and the necessity for online data and internet connectivity. In response to this problem, I developed an offline and privacy-friendly AI chatbot platform known as Atlas AI.

Atlas AI comprises all functionalities in a single platform that runs completely offline. It runs on a user's local machine without depending on external servers and cloud-based technologies. It also does not require user login. Hence, all data remains completely safe and private on the user’s device. The Atlas AI system has two components:

- **Mobile Application**: This serves as the client application that is used to connect to the desktop application over the same Wi-Fi local network.

- **Desktop Application**: This application acts as the processing unit, which runs the AI model on your machine. All computational tasks are performed locally in this application.

One of the key features of Atlas AI is its **offline-first architecture**, which guarantees:

- No internet dependency after initial setup
- No user authentication or account creation
- No data tracking, analytics, or telemetry
- Complete control over data storage using local databases

1.3 Problem Statement

With the growing trend of utilizing AI in various applications, almost all AI-based systems make use of cloud computing technology to handle user requests. Despite their usefulness, there are numerous problems associated with such systems when it comes to issues of privacy, security, requirement of internet connectivity, and lack of user control over their own data.

Current AI-powered chatbots or assistants usually necessitate that users submit their data online for processing. This poses significant risks to personal information because users' data can be stored, processed, and leaked without user permission. Moreover, these types of technologies often require authentication and subscription along with third-party API usage, making them more difficult to implement and use.

One of the most significant problems is the need for internet connectivity at all times. There may be instances when a connection to the internet cannot be achieved for various reasons, thus rendering the technology useless.

Thus, there is a need for an efficient offline AI chatbot platform, which functions entirely using its local resources, maintains total privacy, does not rely on any cloud-based services, and ensures a hassle-free communication environment for both desktop and mobile users.

1.4 Objective of the Project

The major goal of this research project is to design and create an offline version of the AI Chatbot called Atlas AI that runs entirely locally without any dependence on any cloud computing or third party API services.

The main goals of this project can be stated as follows:

1. **To design an entirely offline AI System**

   This system must run independently without any dependence on the internet connection at all times. This will help it perform efficiently in all kinds of environments.

2. **To maintain complete data privacy and security**

   All data, such as conversations and other settings related to the AI, should be stored in the user's device, and none of it should be transferred to external servers.

3. **To maintain complete independence from cloud computing**

   There must be no usage of external APIs and other cloud services. All the functions of the AI should be handled locally through its own built-in models.

4. **To make sure that there is a very intuitive interface**

   The interface of the AI should be intuitive enough to work seamlessly without having much knowledge about it.

5. **To make use of real-time streaming response generation**

   This feature would greatly improve the user experience.

6. **For designing a scalable and sustainable architecture**

   The solution must have a modular architecture using a monorepo strategy that will make it easy for developers to code and scale the solution.

7. **For effective local storage management**

It is imperative to store all conversations and settings on local hardware in an efficient database for quick access.

8. **For easy installation and deployment**

It is important to ensure that the solution can be installed easily without requiring users to go through complex setups.

9. **For optimized performance in local hardware**

It is essential to use all system resources effectively for optimal performance on lower-end devices.

1.5 Scope

The scope of the project focuses on the design and development of **Atlas AI**, a fully offline, privacy-first AI chatbot system that operates entirely on local hardware. The system is intended to provide users with a secure and efficient platform for interacting with Artificial Intelligence without relying on cloud services or external APIs.

The project covers the following aspects:

1. **Offline AI Interaction:** The system enables users to interact with an AI chatbot that runs locally on the desktop, eliminating the need for internet connectivity after initial setup.
2. **Desktop-Based AI Processing**: All core functionalities, including model execution, data storage, and API handling, are performed within the desktop application, ensuring complete control over system operations.
3. **Mobile Integration over Local Network:** A mobile application is provided to enhance accessibility, allowing users to connect to the desktop application through a local Wi-Fi network and interact with the AI remotely.

4. **Local Data Storage and Management:** The system stores all conversations, user preferences, and configurations locally using a database, ensuring fast access and improved data privacy.
5. **Real-Time Streaming Communication:** The chatbot supports streaming responses, enabling users to receive outputs incrementally for a more interactive experience.
6. **User-Friendly Interface:** The project includes intuitive interfaces for both desktop and mobile platforms, ensuring ease of use for both technical and non-technical users.
7. **Model Management Capabilities:** Users can download, manage, and switch between AI models locally, providing flexibility based on system capabilities and user requirements.

CHAPTER 2
LITERATURE SURVEY

Current studies indicate a clear requirement for an AI-based solution capable of functioning offline in areas with poor connectivity because most current solutions, such as ChatGPT and Bard, are based in the cloud, which restricts accessibility and privacy. Current edge computing and RAG studies demonstrate that local processing has numerous advantages.

Table 2.1 Literature Survey

S.No	Paper & Citation	Key Highlights	Limitations	Connection to Atlas AI
1	Li et al., 2022 (IEEE Access) https://doi.org/10.1109/ACCESS.2022.3154860	Proposed an edge-based AI framework for natural language tasks, reducing response latency and bandwidth usage.	Focused on single-domain chatbot applications with limited adaptability.	Supports the concept of deploying lightweight, multi-domain AI models on local devices.
2	Patel et al., 2023 (ACM Computing Surveys) https://dl.acm.org/doi/10.1145/3563764	Reviewed advancements in offline NLP models and retrieval-based conversational systems.	Discussed theoretical approaches without implementation of multi-expert AI assistants.	Reinforces the use of RAG for context-aware, offline AI applications in Atlas AI.
3	Zhang et al., 2023 (IEEE Internet of Things Journal) https://doi.org/10.1109/JIOT.2023.3248125	Presented an IoT-integrated edge intelligence model for data-driven automation in disconnected environments.	Concentrated on IoT sensors rather than conversational AI.	Inspires Atlas AI's edge-computation model for reliable offline performance.
4	Nguyen et al., 2024 (Elsevier Neurocomputing) https://doi.org/10.1016/j.neucom.2024.127816	Developed a RAG-based AI system for medical text retrieval and diagnosis assistance.	Focused only on healthcare, lacking multi-domain adaptability.	Validates Atlas AI's goal to integrate RAG across multiple domains.

5	Kumar et al., 2024 (Springer AI & Society) https://doi.org/10.1007/s00146-024-01892-1	Investigated the ethical challenges of data privacy in AI models using cloud-based infrastructures.	Provided guidelines but no technical solution for on-device privacy.	Strengthens Atlas AI's privacy-preserving offline computation approach.

However, current offline models are often single-domain and face performance limitations. Atlas AI addresses these issues by combining edge-device processing with RAG-based domain knowledge to provide a fast, private, and multi-expert assistant that works without internet.

CHAPTER 3

SOFTWARE REQUIREMENT ANALYSIS

3.1 Problem Statement

Currently, most of the existing conversational systems that employ AI are built using cloud-based infrastructure. These technologies are highly advanced, yet at the same time, they come with several technical difficulties that should be considered.

When speaking about the problems with current software implementation of such technologies, one can mention the following disadvantages:

- Privacy Risk of Data: Since all the user input needs to be sent to the server for further processing, there exists an opportunity for personal data leakage.
- Internet Dependency: Constant connectivity is needed to make this service work, which limits its usage in cases of being offline.
- Latency Issues: Processing of information takes place on external servers, which causes some delay in obtaining the response.
- Integration Complexities: In order to use the majority of available chatbots, users have to configure API keys, set up servers, etc.
- Loss of Local Control: There are only limited ways of controlling how user's data is handled by external systems.

Therefore, the core problem can be defined as:

- Design and develop a software system that enables efficient, secure, and real-time AI-based conversational interaction entirely on local hardware, while ensuring seamless communication between desktop and mobile platforms.
- The system must address key software requirements such as **low latency communication, secure data handling, modular architecture, and ease of use**, while maintaining high performance on local devices.

3.2 Proposed Method

In order to solve the problem of being dependent on the Internet, the problem of security in terms of data privacy, and the issue of limited availability for AI applications in low-connectivity areas, the solution proposed is to create a new application called Atlas AI, which is completely offline and operates only using the power of the device.

3.2.1. Fully Offline System Architecture

The proposed system follows a **completely offline architecture**, where all functionalities including AI processing, data storage, and user interaction are executed locally.

- No internet connection is required during runtime
- All data remains on the user's device

3.2.2. Local AI Model Execution

The system utilizes open-source AI models that run directly on the device. These models process user inputs and generate responses without relying on remote servers. This provides:

- Faster response times (low latency) and Enhanced data privacy
- Independence from cloud infrastructure

3.2.3. Retrieval-Augmented Generation (RAG) with Local Data

To improve response quality, the system implements a **local Retrieval-Augmented Generation (RAG)** approach:

- Domain-specific datasets are stored locally
- Relevant information is retrieved based on user queries and combined with AI-generated responses

This ensures:

- Context-aware responses
- Improved accuracy and domain-specific performance

3.2.4. Domain-Specific Expert Modes

The system supports multiple specialized modes to handle different use cases:

- Health Advisor
- Sports Analyst
- Concept Explainer
- Agriculture expert

3.2.5. Edge Device Computation

All computations are performed on the user's device (edge computing), ensuring:

- Low latency performance
- High reliability in offline environments

3.2.6. Local Data Storage and Management

The system uses a local database to store:

- Chat history
- User preferences
- Domain datasets

This enables complete user control over information

3.2.7. User-Friendly Interface

The application provides an intuitive interface that allows users to interact with the system easily. It is designed to be accessible for both technical and non-technical users.

The proposed method ensures a **secure, efficient, and completely offline AI system** by integrating local AI processing, RAG-based enhancement, and domain-specific intelligence. By eliminating internet dependency, Atlas AI provides a reliable and privacy-focused solution suitable for real-world deployment in low-connectivity environments.

CHAPTER 4

SOFTWARE DESIGN

The analysis of software plays a crucial role in the project. With the thorough analysis of software, it becomes possible for us to determine the main objectives and choose the right technology and tools for developing the system. We can ensure that the designed system would be effective for all users.

4.1 Data Flow Diagrams (DFDs)

Data Flow Diagrams (DFDs) are visual tools used to show the movement of data within a system. They help us understand how information enters the system, how it is processed and where it goes.

DFD Level 0:

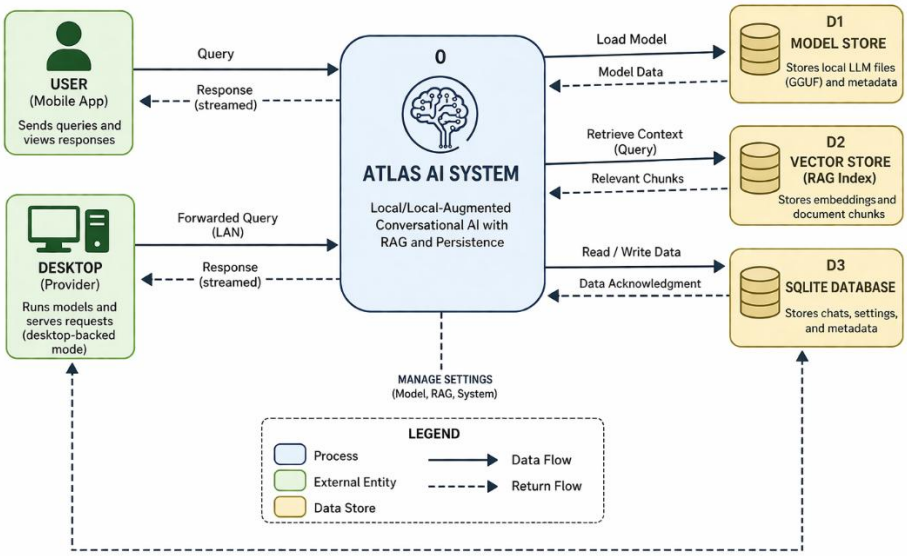

Fig 4.1.1: DFD Level 0

- The diagram shows **Atlas AI System** as the central process that receives user queries, interacts with the desktop provider, and manages model loading, context retrieval, and local data storage.
- It illustrates **data flow**, where responses are generated using local models, RAG-based context retrieval, and SQLite persistence.

DFD Level 1:

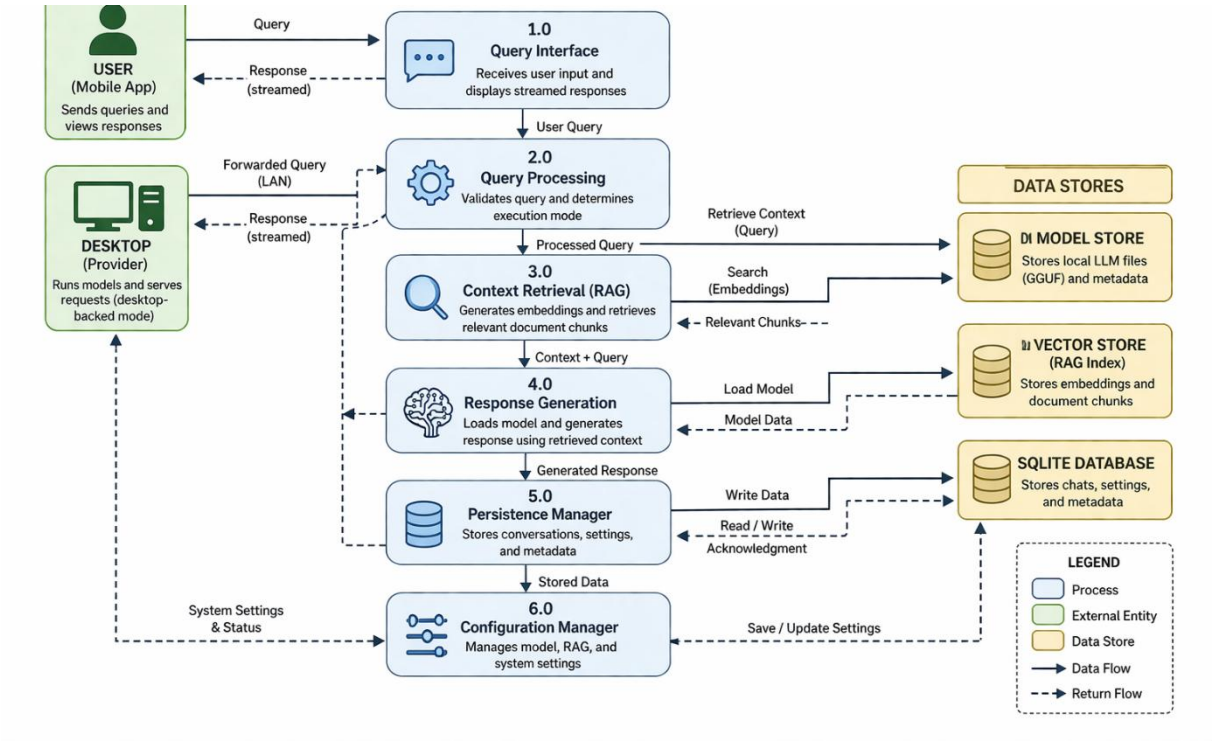

Fig 4.1.2: DFD Level 1

- The DFD Level 1 diagram breaks the Atlas AI system into major internal processes such as query interface, query processing, context retrieval, response generation, persistence management, and configuration management.
- It illustrates the detailed flow of data between internal modules and data stores such as the model store, vector store, and SQLite database, highlighting how Atlas AI generates context-aware responses.

4.2 UML DIAGRAMS

4.2.1 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor.

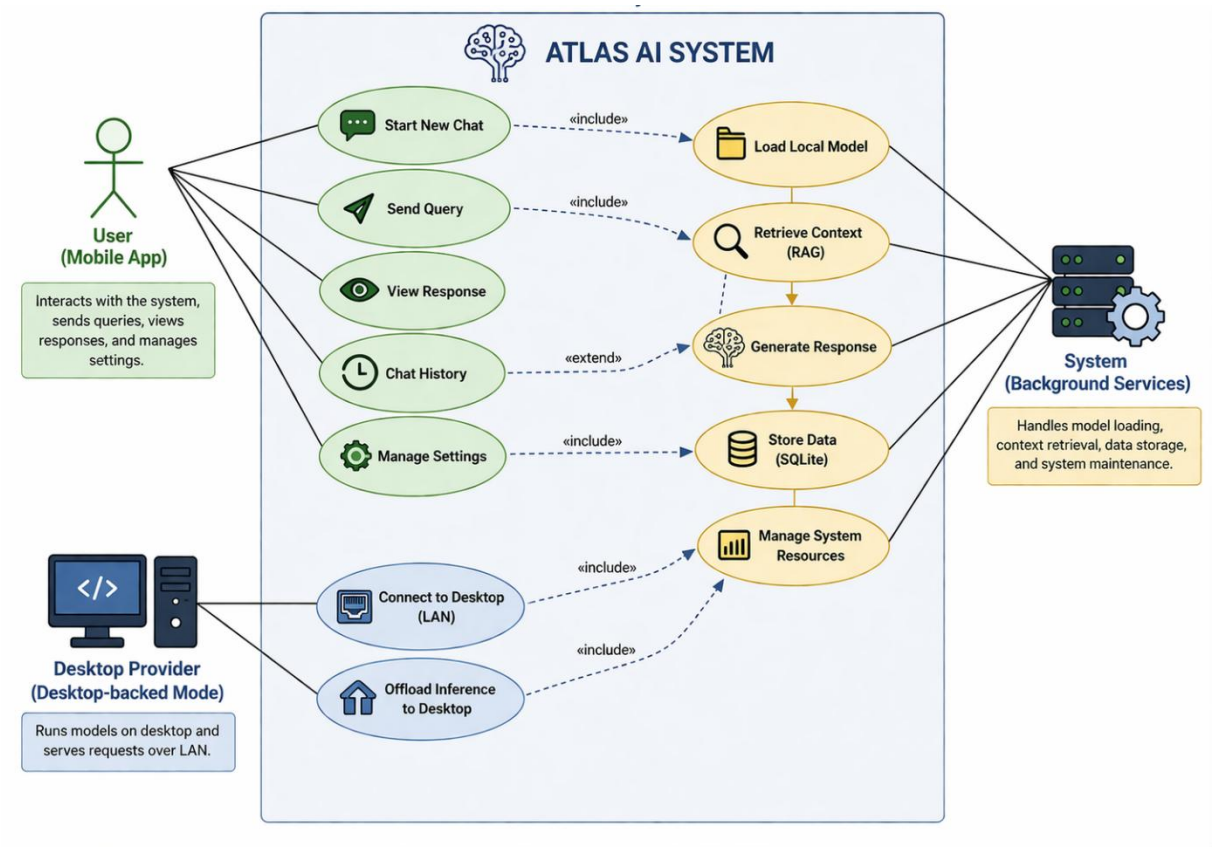

Fig:4.2.1: Use Case Diagram

- The use case diagram shows the User as the primary actor who interacts with the Atlas AI system to start a new chat, send queries, view responses, access chat history, and manage settings.
- It also illustrates how Atlas AI supports desktop-backed execution, where the desktop provider can connect through LAN and assist the system in handling inference and managing system resources.

4.2.2 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods) and the relationships among the classes. It explains which class contains information.

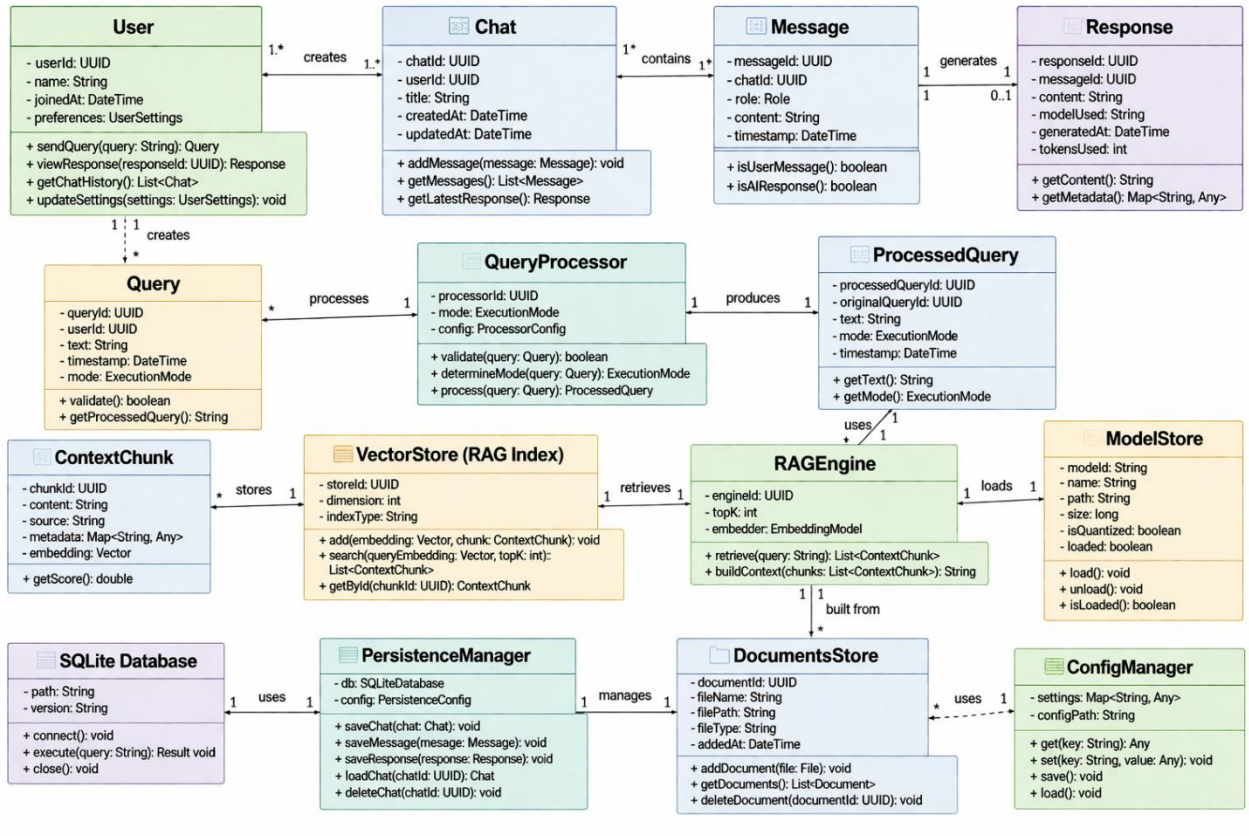

Fig 4.2.2: Class Diagram

- The class diagram represents the structural design of Atlas AI by showing the major classes such as User, Chat, Message, Query, QueryProcessor, RAGEngine, ModelStore, and PersistenceManager along with their attributes, methods, and relationships.
- It illustrates how Atlas AI organizes query handling, context retrieval, response generation, model management, and data persistence through interconnected classes, thereby reflecting the modular and object-oriented architecture of the system.

4.2.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart.

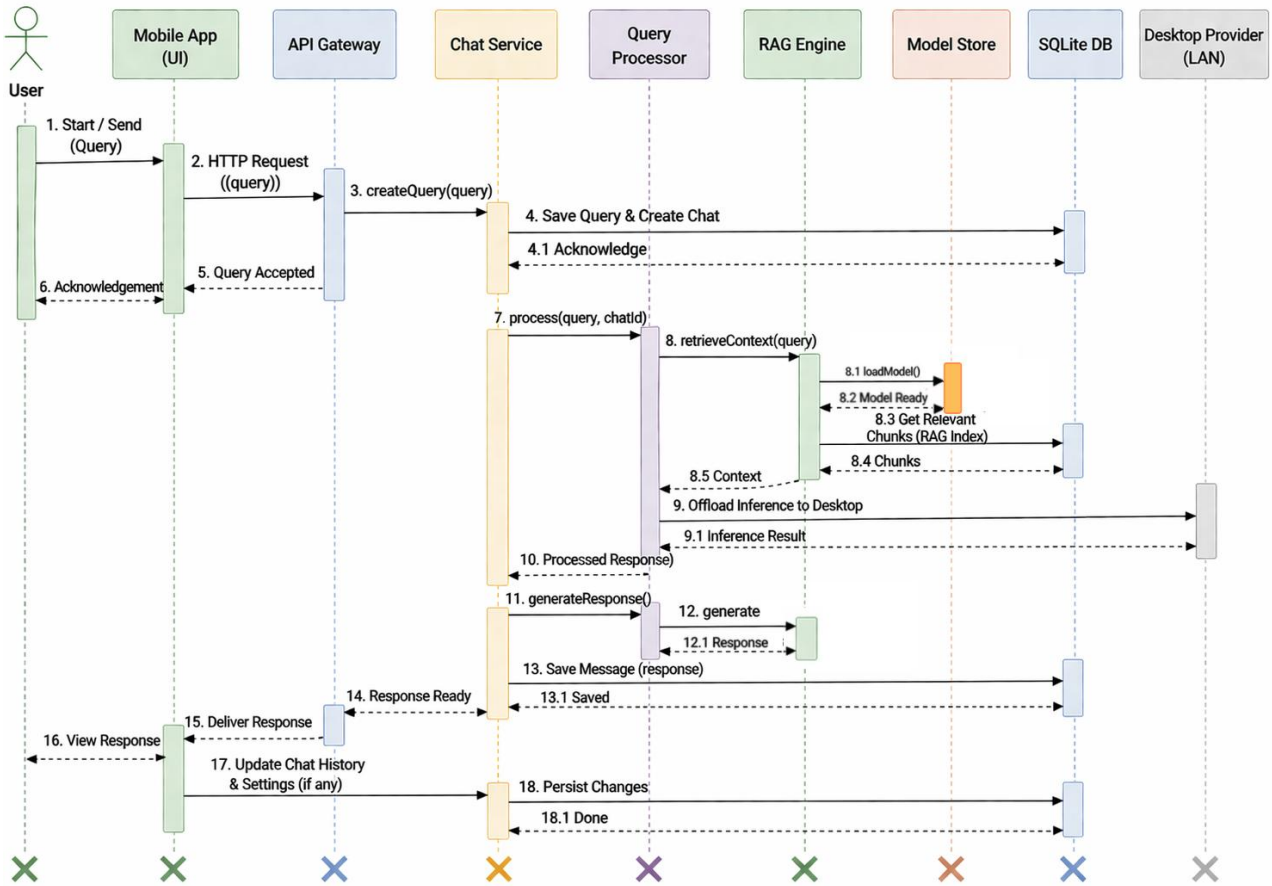

Fig 4.2.3: Sequence Diagram

- The sequence diagram illustrates the step-by-step interaction flow from the user sending a query to the system processing it through components like the mobile app, query processor, RAG engine, and model runtime before generating and returning a response.
- It clearly shows the coordination between retrieval, model inference, and data persistence, including optional desktop-backed execution, highlighting how Atlas AI ensures efficient response generation and storage in an offline-first architecture.

4.2.4 COLLABORATION DIAGRAM:

In collaboration diagram the method call sequence is indicated by some numbering technique as shown below. The number indicates how the methods are called one after another. The method calls are similar to that of a sequence diagram. Collaboration diagram shows the object organization.

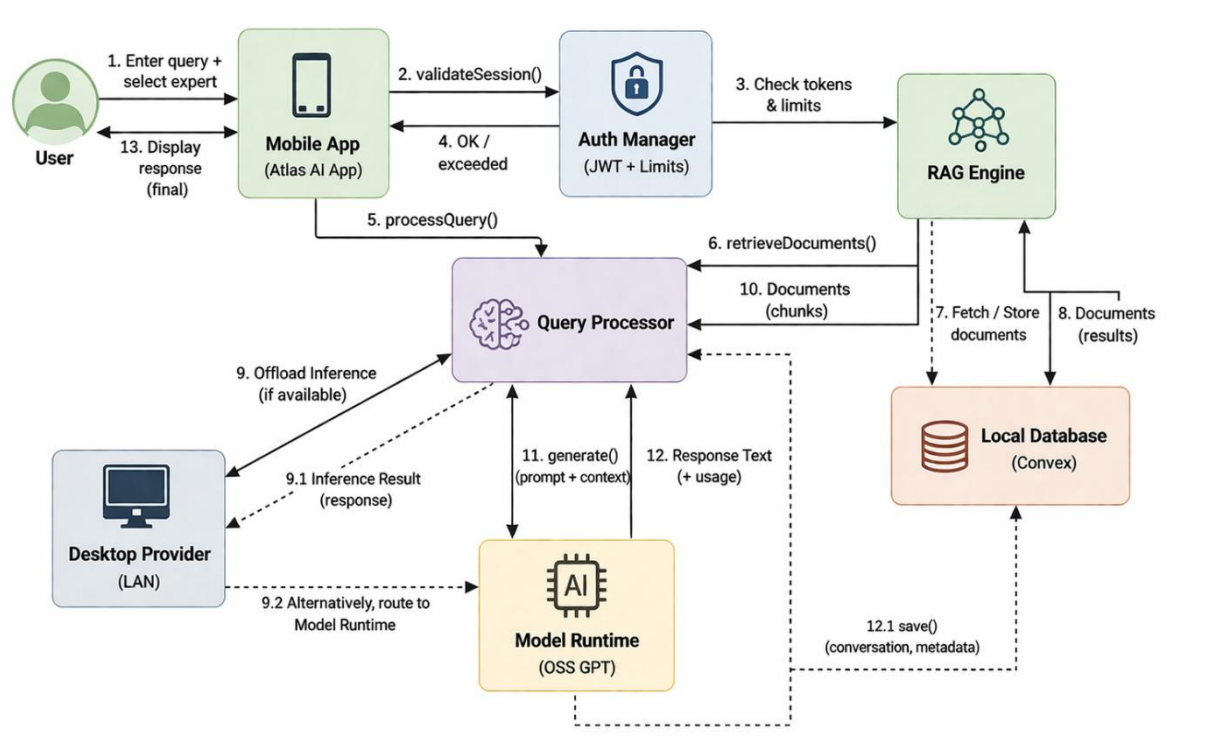

Fig 4.2.4: Collaboration Diagram

- The collaboration diagram shows how different components such as the mobile app, auth manager, query processor, RAG engine, model runtime, and database interact with each other to process a user query and generate a response.
- It highlights both primary and optional flows, including desktop-assisted inference, demonstrating how Atlas AI coordinates multiple modules to ensure efficient processing, context retrieval, response generation, and data persistence.

4.2.5 DEPLOYMENT DIAGRAM:

Deployment diagram represents the deployment view of a system. It is related to the component diagram. Because the components are deployed using the deployment diagrams.

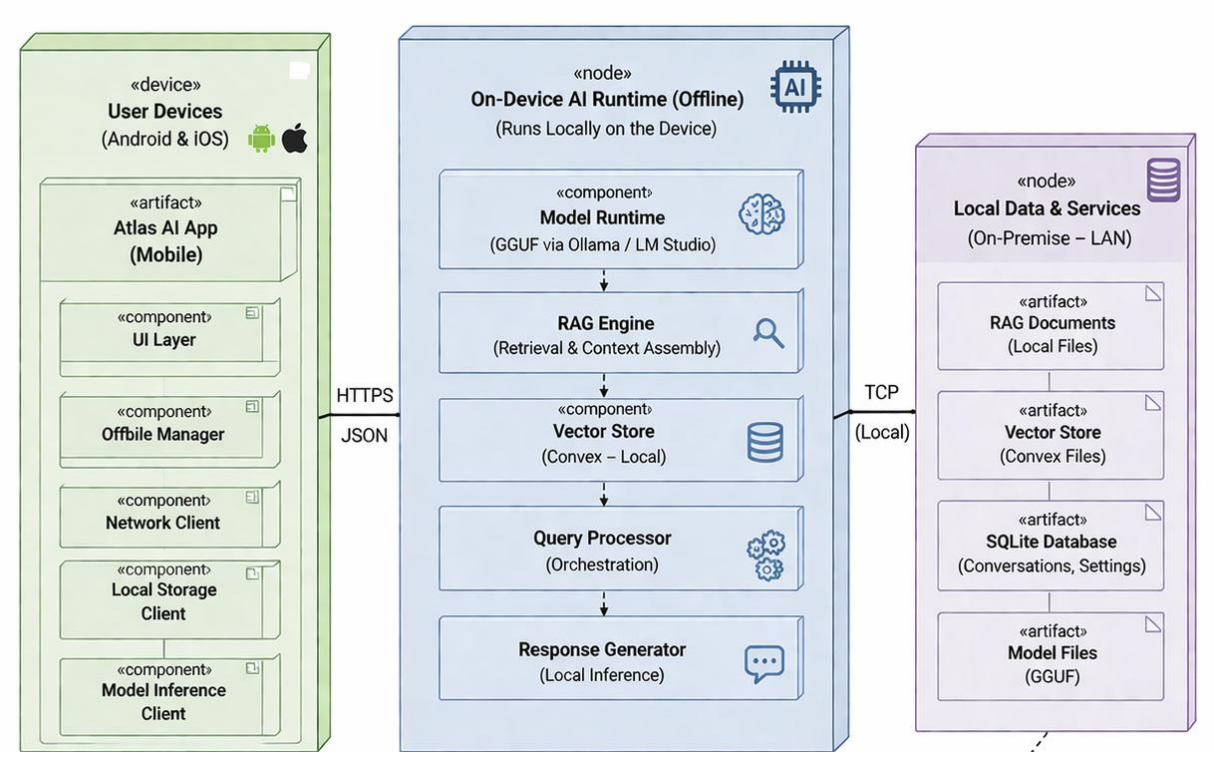

Fig 4.2.5: Deployment Diagram

- The deployment diagram represents the physical architecture of Atlas AI, showing how components are distributed across user devices, on-device runtime, and local data services, all operating within a LAN-based offline environment.
- It highlights that core functionalities such as model inference, retrieval, and data storage are executed locally on the device, ensuring privacy, reduced latency, and independence from external cloud infrastructure.

4.2.6 ACTIVITY DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

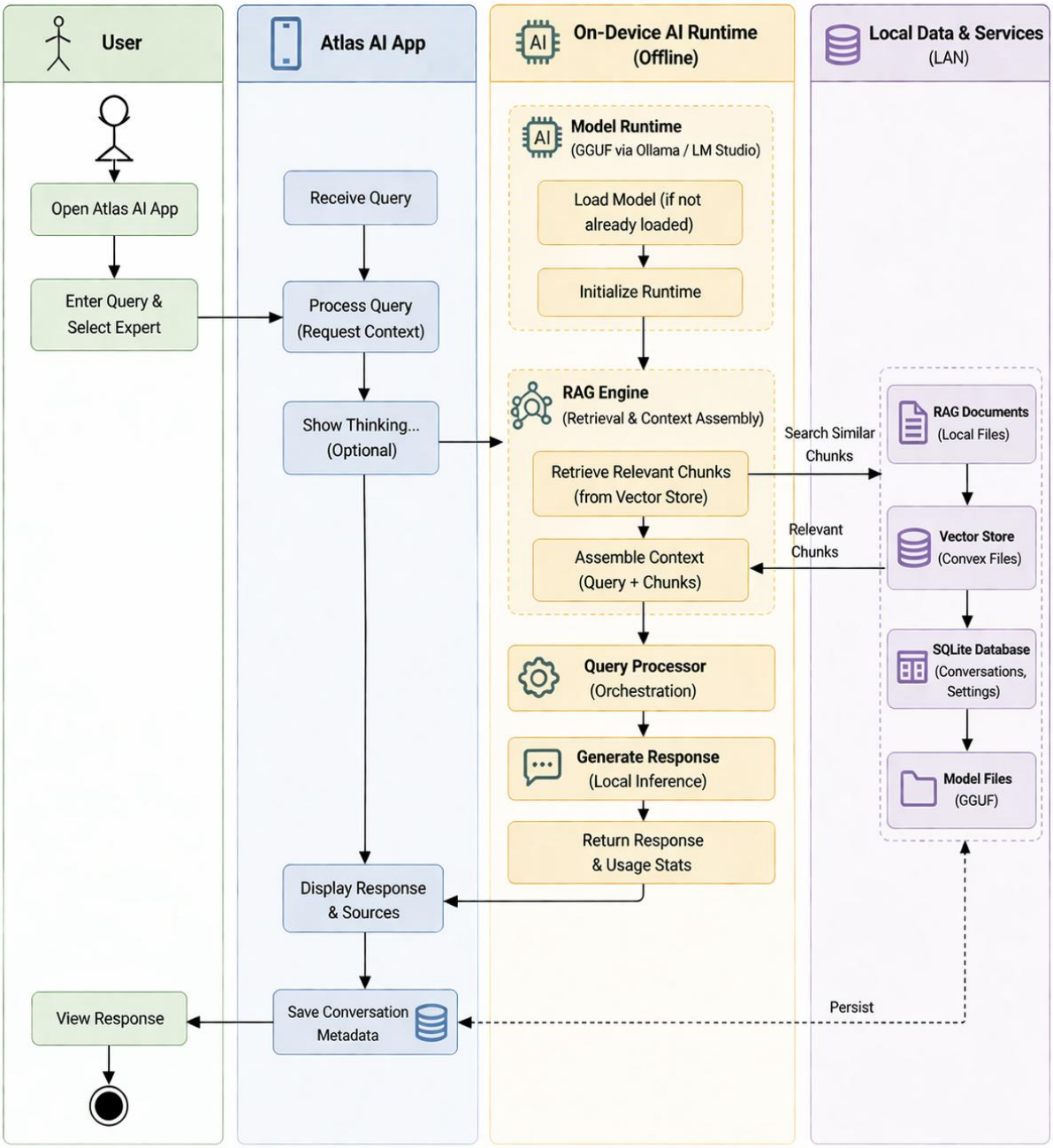

Fig 4.2.6: Activity Diagram

- The activity diagram illustrates the workflow of Atlas AI from user query input to response generation, showing how the system processes queries through the app, on-device runtime, and local data services.
- It highlights the sequential flow of actions including query handling, context retrieval using RAG, local model inference, and persistence of conversation data, emphasizing a fully offline and streamlined execution process.

4.2.7 COMPONENT DIAGRAM

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development.

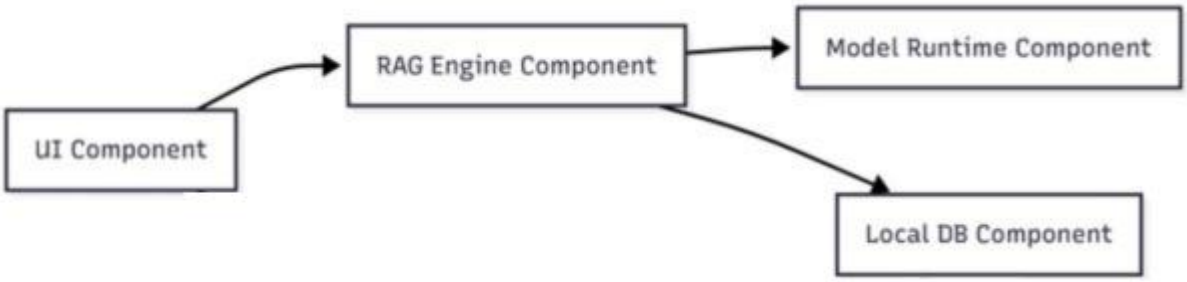

Fig 4.2.7a Component Diagram

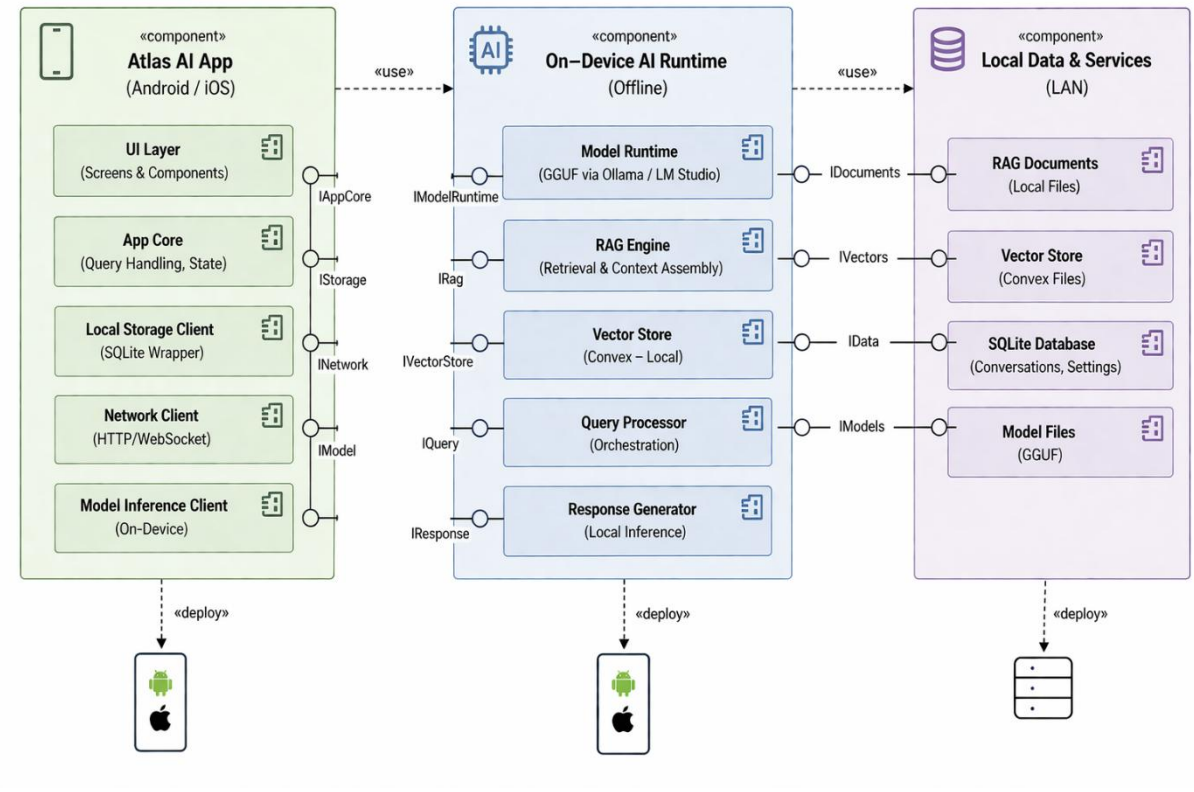

Fig 4.2.7b Component Diagram

- The component diagram represents the modular structure of Atlas AI by showing how the mobile app, on-device AI runtime, and local data services are divided into

independent components such as UI layer, query processor, RAG engine, vector store, and response generator.

- It highlights the interaction between components through defined interfaces and dependencies, demonstrating how data flows from the application to local processing modules and storage systems while maintaining a fully offline architecture.

4.2.8 State Chart Diagram:

A State Machine represents the different states that a system, process, or object can be in at various points in time. These states are interconnected through transitions, which depict how the system moves from one state to another based on specific conditions, events, or triggers.

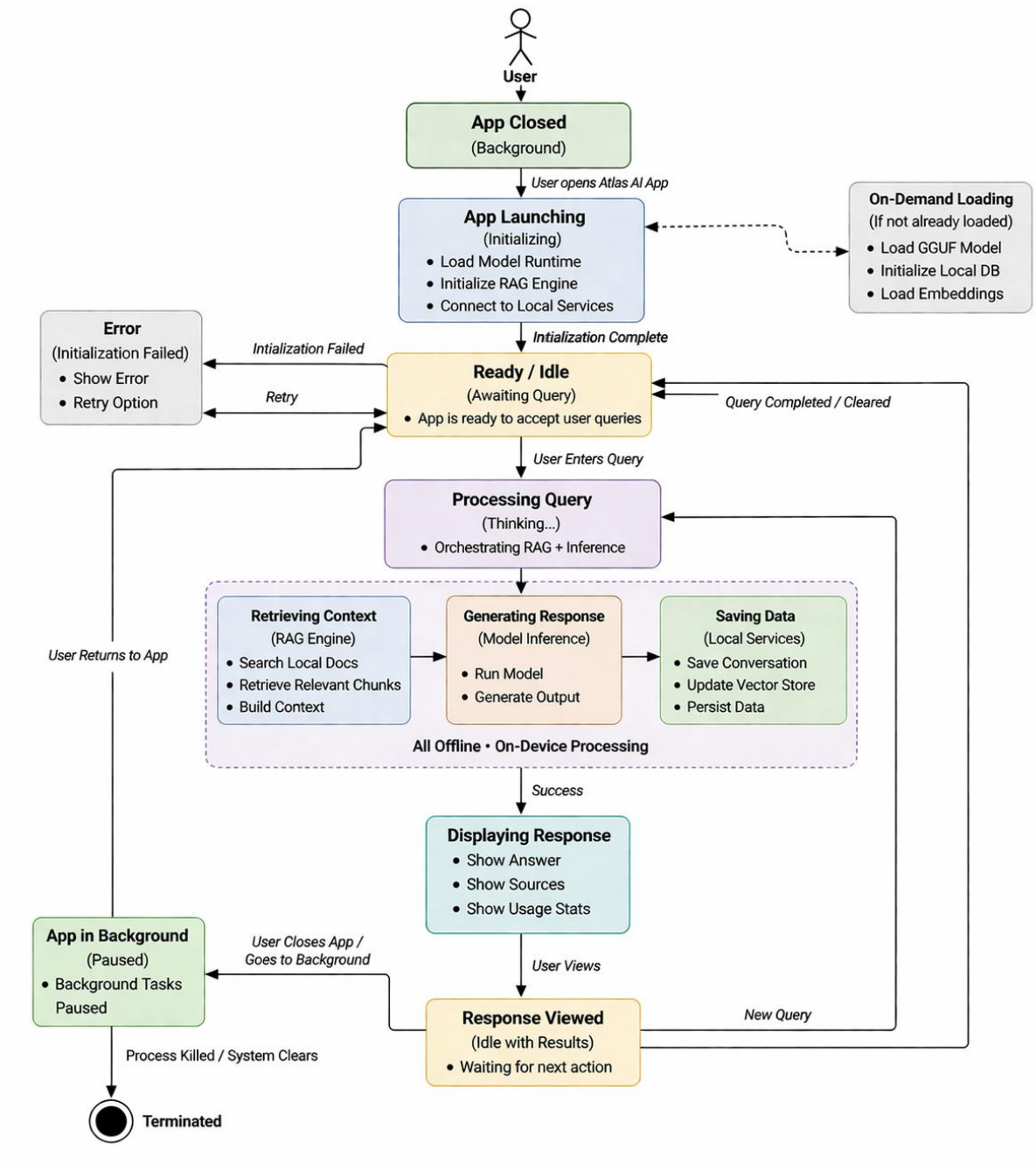

Fig 4.2.8: State Chart Diagram

- The state chart diagram represents the different states of the Atlas AI application, such as app closed, launching, ready, processing query, displaying response, and background, along with transitions triggered by user actions and system events.
- It highlights how the system moves between states during query processing, including handling initialization, inference, response generation, and persistence.

4.2.9 Object Diagram

An Object Diagram is like a snapshot of the system at one point in time. It shows the real objects that exists. This diagram is useful because it gives you a clear picture of the system's structure, making it easier to understand how different parts of the system are working together.

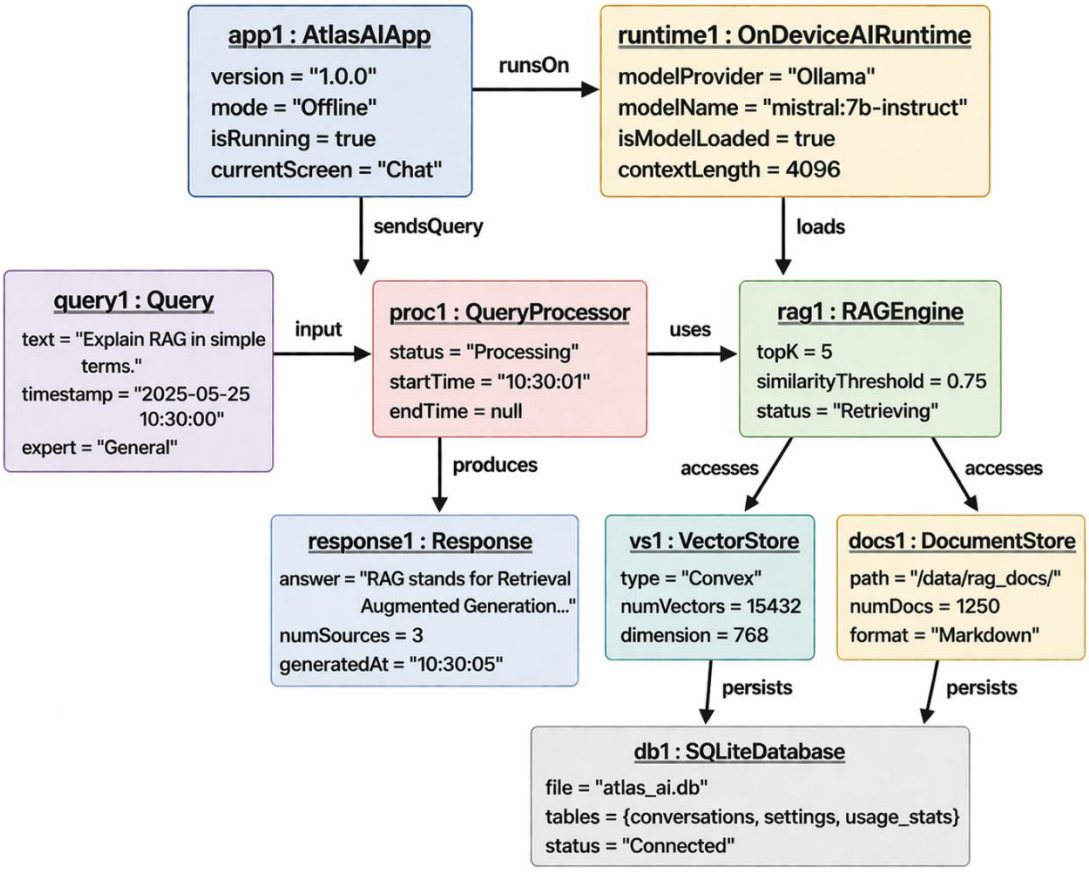

Fig 4.2.9: Object Diagram

- The object diagram represents a snapshot of the Atlas AI system at runtime, showing specific instances such as the app, query processor, RAG engine, vector store, and database along with their current states and attribute values.

- It illustrates how these objects are interconnected during query processing, highlighting real-time relationships between components involved in context retrieval, response generation, and data persistence in an offline environment.

4.3 E-R Diagrams

The Atlas AI database consists of three tables stored in a single SQLite file (atlas.db). The conversations table is the core entity, holding metadata for each chat session including the model used, and timestamps for creation and last activity. The messages table is a child of conversations each message belongs to exactly one conversation via a foreign key with cascade delete, ensuring messages are automatically removed when their parent conversation is deleted. The role column distinguishes between user and assistant turns. The settings table is a standalone key-value store with no relationships, used to persist desktop-wide configuration such as the default model. All data access goes through synchronous better-sqlite3 helpers in server/db.ts, with WAL mode enabled for better read performance during streaming.

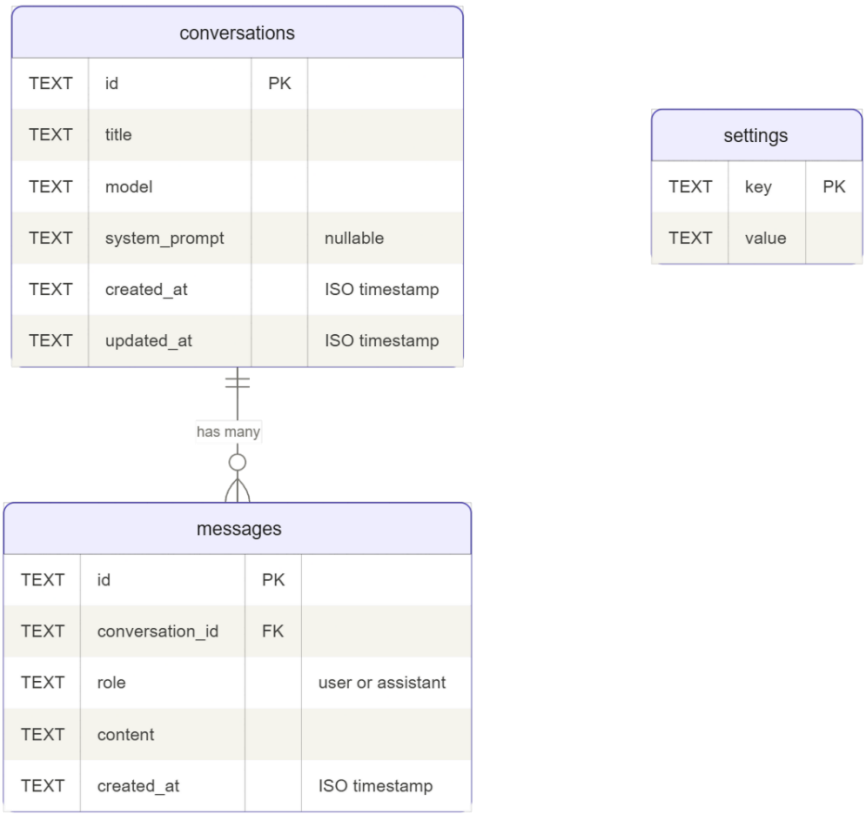

Fig 4.3.1 E-R Diagram

CHAPTER 5

SOFTWARE & HARDWARE REQUIREMENTS

5.1 Hardware Requirements

- **Processor:**
  - Desktop: Intel i5+ / Ryzen 5+
  - Mobile: Snapdragon 7+ Gen / Apple A14+
- **RAM:** 8-12+ GB.
- **Storage:** 512+ GB
- **AI Accelerator:** Desktop GPU or NPU
- **Internet Connection:** Optional but recommended to connect in a while for updates and sync

5.2 Software Requirements:

- **Operating System:** Android 10+ / iOS 15+ and Windows 10+ / macOS / Linux
- **Frontend:** React Native / Expo, Next.js, Electron, Tailwind CSS, ShadCN
- **Backend:** Fastify, Python, Ollama, llama.cpp and llama.rn
- **Database/File Storage:** AsyncStorage, Sql-lite
- **Libraries:** Transformers, Zustand, FileSystem,
- **Models:** llama-3.2, Qwen-2.5, Phi-3, Gemma

5.3 Languages Used

- **JavaScript / TypeScript:** Mobile app, state management, Desktop Management
- **Python:** Preprocessing, Whisper, and embedding tasks, RAG pipeline, Ollama communication, API

CHAPTER 6

MODULES

These are the modules used in our project:

1. Mobile Client Module
2. API Server Module (Fastify)
3. AI Processing Module (Ollama)
4. Database Module (SQLite)
5. Desktop Core Module

6.1 MOBILE CLIENT MODULE

The purpose of the Mobile Client Module is to facilitate an easy-to-use and transportable means of engaging with the AI system Atlas. Developed in Expo and React Native, this module can be run on both Android and iOS devices, making it easily accessible. Users have the choice of accessing AI capabilities either through connection to the desktop system via local network or by carrying out inferences on their device.

The mobile client can work in two different modes; known as providers. First, the desktop provider involves connection of the mobile client via local Wi-Fi network to the Fastify API server on the desktop system. In this case, the mobile client sends query data from the user to the server and receives streamed answers from the server. However, all processing and storage happens on the desktop server, while the mobile client is just used as an interface.

The second mode of operation is the local provider; here, the application performs AI inference on-device using GGUF models. The implementation of this mode uses a native bridge that utilizes llama.cpp library functions. The chosen model will be saved to the application's sandbox through the file system. There will be no need for a desktop computer connection.

The user interface submodule provides a chat-based interface to communicate with the agent. This submodule supports sending queries, viewing answers, managing conversations,

and switching providers. The intuitive design of this submodule makes it accessible for users regardless of their level of technological knowledge.

The provider management module is designed to support seamless switching between the desktop and local modes of operation. It stores configurations such as the IP address of the desktop computer, the chosen model, and its connection state. Users can choose the best mode depending on their current conditions and device availability.

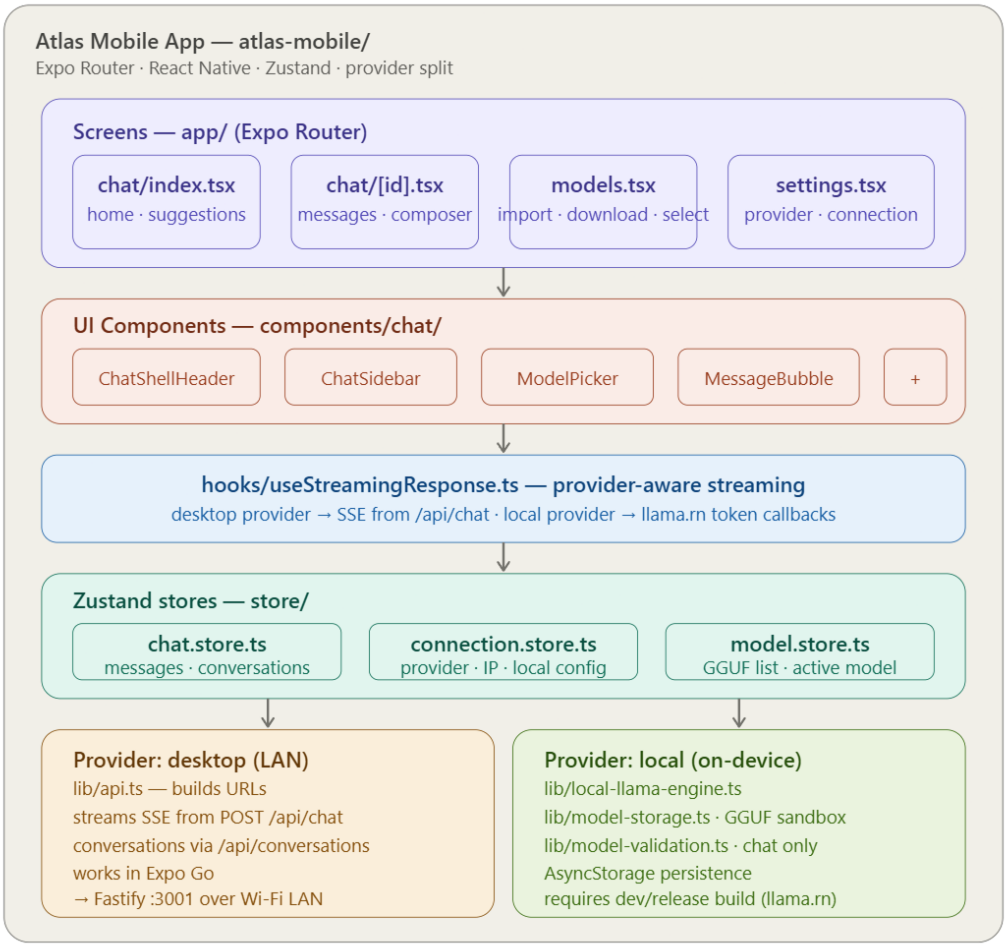

Fig. 6.1.1 MOBILE CLIENT MODULE

The desktop communication submodule communicates with the Fastify API server running on the desktop computer when using desktop mode. In particular, this submodule is responsible for requesting HTTP endpoints like /api/chat, handling SSE events, and receiving the information about models and the system status from the server.

The local inference submodule is responsible for executing AI models directly on the mobile device. It loads GGUF model files, applies performance configurations based on device capability, and generates responses. It also includes validation mechanisms to ensure that only compatible models are used.

The state management module, implemented using Zustand, manages application state efficiently. It handles conversation data, connection status, selected provider, and user preferences, ensuring smooth interaction across the application.

The local storage module uses AsyncStorage to persist conversation data and settings when operating in local mode. Additionally, the file system is used to store GGUF model files securely within the application environment.

Overall, the Mobile Client Module enhances the accessibility and flexibility of the Atlas AI system by providing both remote access via LAN and fully local AI capabilities on the device. Its modular and efficient design ensures a seamless user experience while maintaining the core principles of offline functionality and data privacy.

6.2 API SERVER MODULE (FASTIFY)

The API Server Module which has been developed using Fastify acts as a key point in the communications of the entire system. This is because it is a local HTTP server that facilitates all interactions of the users' interfaces (both desktop and mobile versions) with the AI processing part. This makes sure that all processes take place in an effective way.

Initialization of this server takes place on the desktop version. The server works by listening to a specified port number, which is usually set at 3001. Besides, it listens on all interfaces (0.0.0.0), hence giving room for accessing the server through localhost and the LAN. This is why the idea behind considering the desktop as the LAN brain becomes clear.

An important task that is implemented within this module involves the management of API endpoints. There are various REST endpoints defined within this module. There are also other endpoints that allow streaming responses. The server provides endpoints for health

monitoring, conversation management, chatting, models manipulation and settings. These endpoints are designed in such a way that each of them performs certain tasks.

Another important task that is implemented within this module is the chat processing. The processing of the incoming requests is performed in such a way that after validation of the request and adding additional prompts, if required, the server sends the processed query to the AI runtime (Ollama). Then, the server processes the response provided by the Ollama and sends it to the user. In addition, the chat processing module takes care of managing conversations' contexts.

In this module there is also implemented Server-Sent Events mechanism which allows performing streaming of responses. In fact, rather than providing a single response, the server sends data in tokens. That is, each token is transmitted as an event. This mechanism allows displaying responses to user's requests in a more interactive manner.

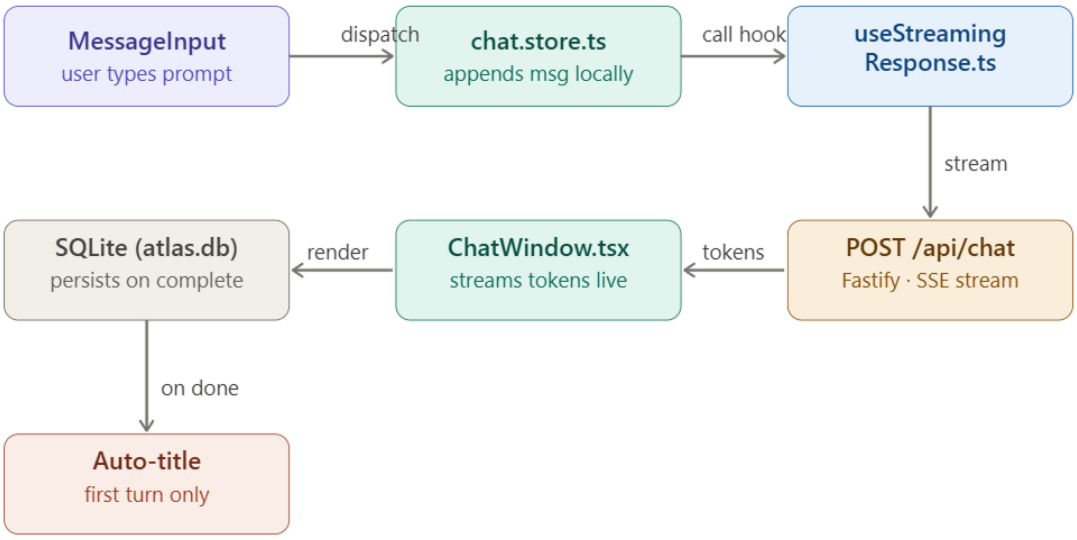

Fig 6.2.1 API SERVER MODULE

In addition to the above modules, the module also contains a chat conversation/persistence handler, which works with the local SQLite database. This module stores messages from users, responses from assistants, chat metadata, and timestamps. All

operations with the database are consolidated on the server side, guaranteeing consistency and making the server the only source of truth.

There is also a model management submodule, which provides a connection to the Ollama runtime engine. Users can manage their models by listing, downloading (and tracking download progress), and removing them from the system. Each operation is provided via an API endpoint and is done asynchronously.

The networking and LAN communications module ensures that the server is accessible via the local area network. The mobile device can communicate with the desktop computer via the LAN IP address of the server. The server is set up with very relaxed CORS policies to enable internal communications while not exposing itself to public networks.

The API Server Module is essential in coordinating the system operation, facilitating instant communication, and optimizing data exchange between components. Being lightweight, modular, and highly performant, this module is suitable for offline-first AI applications.

6.3 AI Processing Module (Ollama)

The AI Processing Module, which is run using Ollama, is one of the main computing components of the Atlas AI framework. Its functionality includes execution of local Large Language Models (LLMs) and providing intelligent responses based on inputs from users. This way, all processes involving AI are conducted locally on the desktop computer and hence no data privacy is violated, and reliance on remote servers is not required.

Underpinning the AI Processing Module is the Ollama runtime. It acts as a local model server on the desktop computer, which can be accessed via the URL "http://localhost:11434". Meanwhile, Fastify API server interacts with the runtime to conduct all processes involved in models like chat inference, list, download, and delete actions.

This submodule performs the task of processing the user requests. Once the request is made by the user, the message is delivered from the API server to the Ollama runtime in combination with the conversation context. Based on this information, the model performs reasoning and gives an answer. This submodule allows for multi-turn conversations by managing context based on the message history.

One important aspect of this submodule is related to the ability to generate responses via token streams. Rather than generating the whole response in one go, the Ollama runtime creates output incrementally in the format of tokens. These tokens are sent in streams to the API server, whereafter they are passed to the client's browser via Server-Sent Events. This significantly increases performance and creates better user experience as the output can be shown right away.

This submodule deals with the interactions between the system and any available AI models. It provides the following functions:

- Listing installed models
- Pulling (downloading) new models
- Removing existing models

These functions are crucial in ensuring that users have the option to flexibly choose models depending on the processing power of their devices.

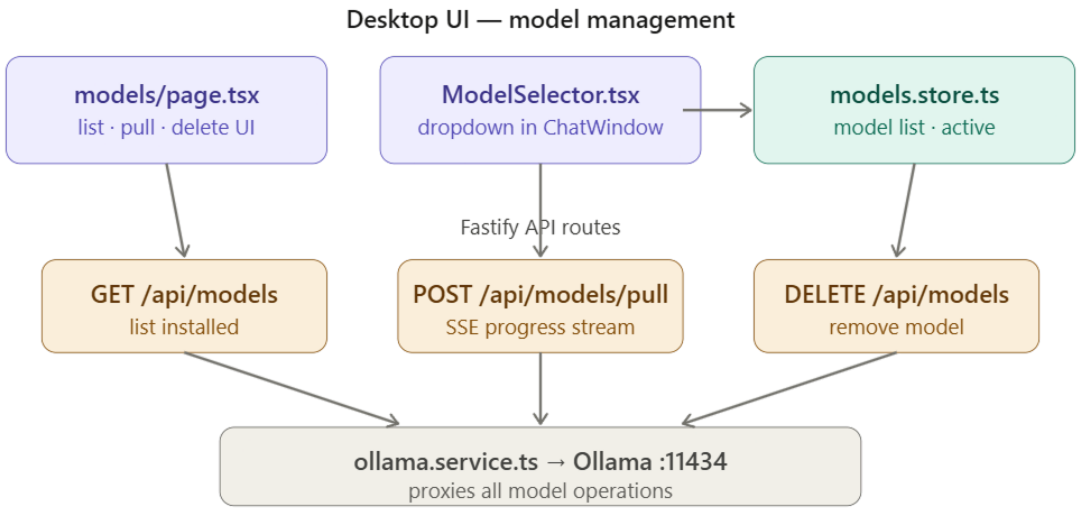

Fig 6.3.1 AI Processing Module

The Response Aggregation Submodule is responsible for aggregating the stream of tokens to form an appropriate response. Although tokens are streamed to offer real-time responses to users, the final responses are assembled and saved to the database. In this way, there will be uniformity between the output presented to the user and what is stored by the system.

In addition to its core functionality, this submodule offers several other functionalities such as automatically generating titles for conversations and managing system prompts for models' use. This means that the module not only increases the functionality of the system but also improves its intelligence levels.

From the above analysis, it is clear that the AI Processing Module is vital in offering offline and intelligent conversation capabilities. Since this process relies on the local processing of models using Ollama, it results in high performance, fast speed, and full data privacy.

6.4 Database Module (SQLite)

Database Module, which is made on SQLite technology, is a module that manages the persistence layer and all the data associated with user interactions, configurations, and system data in general. This module is used to manage the storage of data on a local disk of a user. As Atlas AI operates based on the offline-first model, this module becomes very important in this case because we need to ensure the existence of data even in the case when we have no access to third-party storages.

The database is kept as a file called atlas.db in an app folder of the system. This database is being used only by the desktop application via the API server. Better-sqlite3 library is used for synchronous and fast database operations in a desktop environment.

The submodule of database initialization is used for creating the database at startup time and checking if all the necessary tables are present. Also, all necessary configurations of the database are set including the usage of Write-Ahead Logging mode and enabling foreign keys.

Schema & Table Management submodule includes definition of a database schema which in this case includes the following three tables:

- **Conversations Table**: Stores metadata about each conversation, including ID, title, model used, timestamps, and system prompts.
- **Messages Table**: Stores individual messages associated with conversations, including role (user/assistant/system), content, and timestamps.
- **Settings Table**: Stores application configuration settings as key-value pairs.

The **conversation management submodule** handles operations such as creating, retrieving, updating, and deleting conversations. It ensures that conversations are organized efficiently and displayed in the correct order based on recent activity.

This submodule is charged with storing messages received in chat interactions. It is able to link the conversations to messages using foreign keys in the form of relations. The messages will be stored in a sequential fashion as part of maintaining context necessary for the AI algorithm.

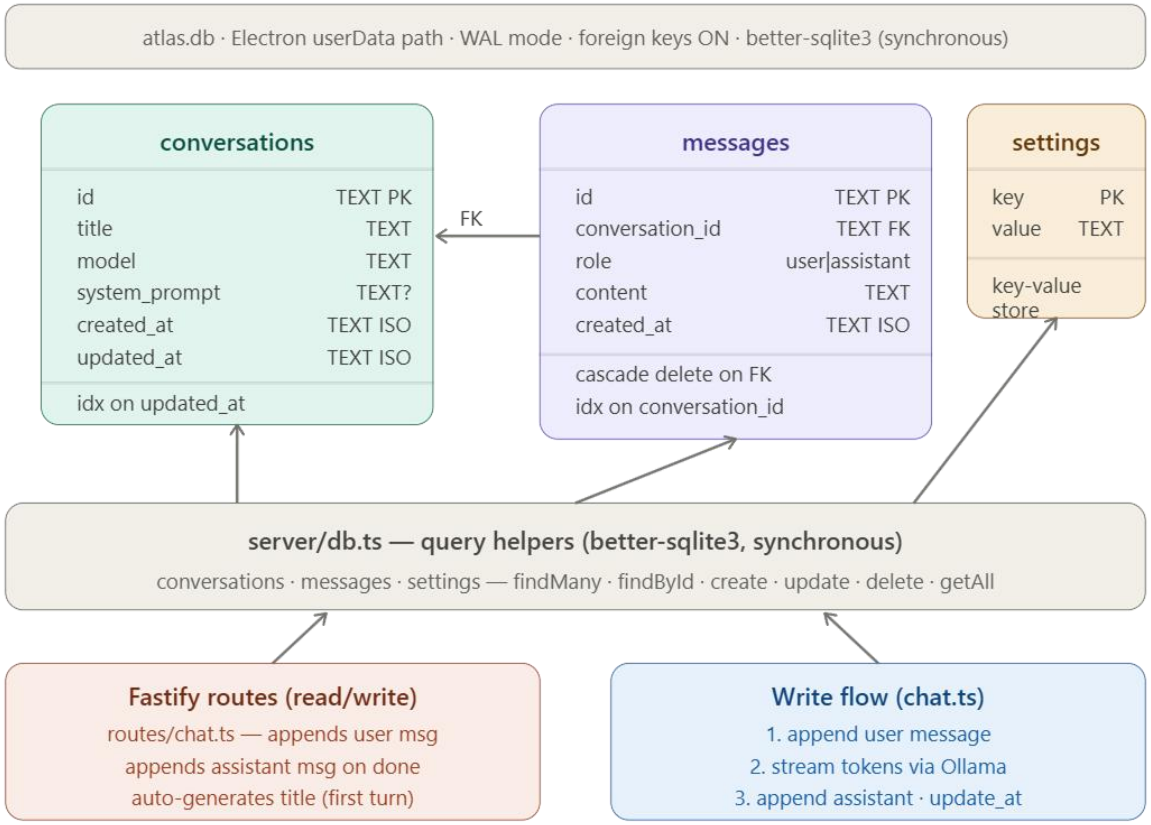

Fig. 6.4.1 Database Module (SQLite)

The Settings Management Submodule is used for setting up persistent application-level configurations including choice of default model, system prompts, and other user-defined settings. They can later be accessed and updated using API endpoints.

Indexing of frequently accessed fields such as conversation identifiers and timestamps is used in order to optimize queries and ensure efficient data retrieval despite growing data volumes.

In summary, the Database Module serves as an effective means for persistent data storage on a local device level. All information about users will remain localized to the same device in line with the project requirements for performance, security, and privacy.

6.5 DESKTOP CORE MODULE

The Desktop Core Module serves as the backbone of the entire Atlas AI application. It consists of all the necessary components that help initialize and coordinate all other modules in the system. In its architecture, the desktop core module relies on Electron, providing a convenient cross-platform desktop environment for the application.

At the core of this module is Electron Main Process, responsible for performing various system-level actions like initialization, managing processes, handling resources, and creating windows in the application. In particular, it is tasked with the launch of all critical services, including the local API server and the AI runtime environment. All backend services are launched in a proper order to ensure system stability.

Renderer Process is a crucial part of this module because it implements the GUI functionality of the application using Next.js framework. Using the renderer process, users can communicate with the system via the chat interface, sending their requests, getting answers from the system, managing dialogues, and changing settings.

Frontend communicates with the backend through IPC. The IPC mechanism is essential to ensure safe interactions between components and prevent any uncontrolled accesses to the system resources from the renderer process.

Other vital submodules include those for running the application in the system tray and handling window management, which makes it possible for the application to continue running in the background and accessing it through the system tray. This makes the application more user-friendly since users can minimize it without interrupting its essential processes.

This desktop module makes sure that all other desktop components, including API server, database, and AI engine, are well connected and work together as expected. The desktop

core module functions as a control hub for coordinating the operation process of the components.

In conclusion, this module is an important module for ensuring the proper working of the Atlas AI system because it serves as the link between the user and the internal components of the system.

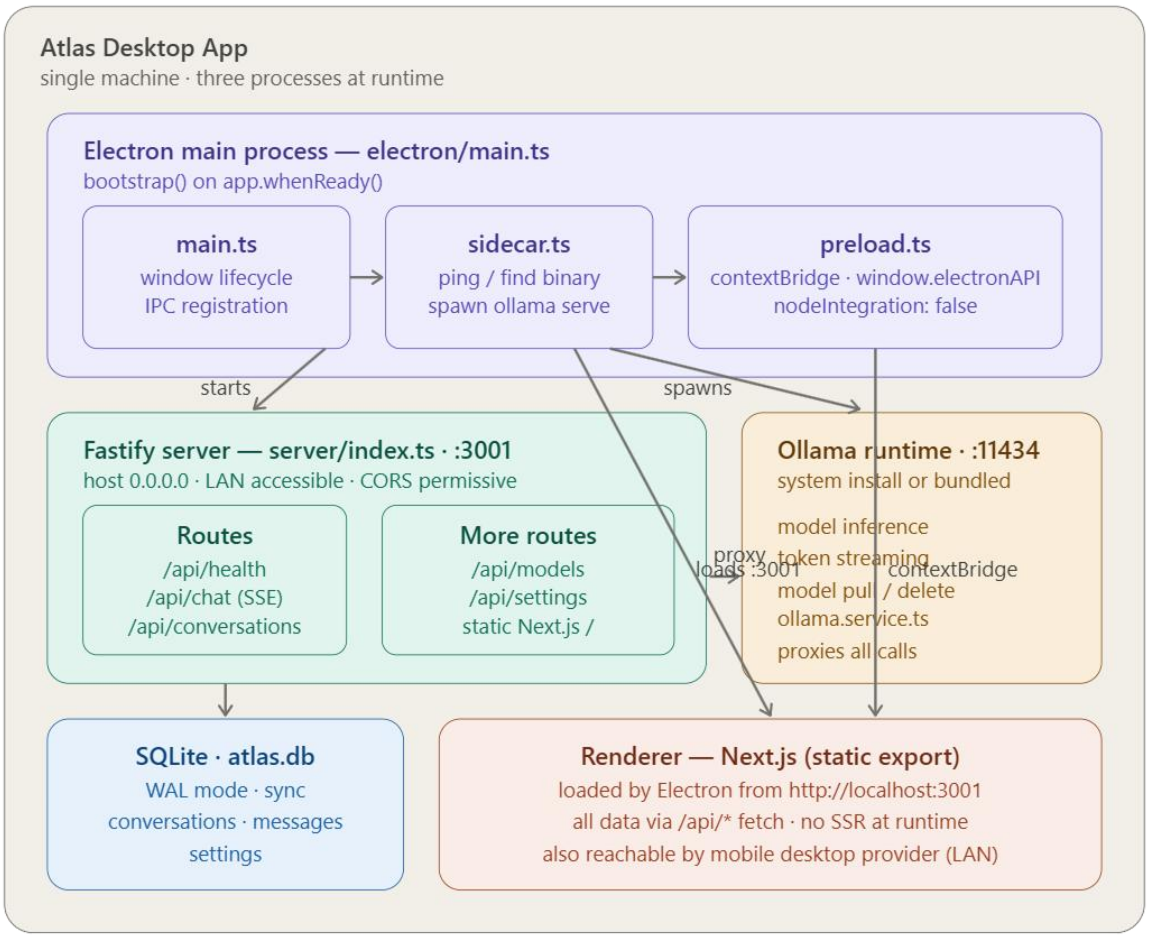

Fig. 6.5.1 DESKTOP CORE MODULE

CHAPTER – 7

Implementation

7.1 Technologies Used

The Atlas AI system is implemented using the following technologies:

- **TypeScript / JavaScript** – Core programming languages used for both frontend and backend development
- **Electron** – Used to build the desktop application with native system access
- **Next.js** – Used for developing the desktop user interface (renderer process)
- **Fastify** – High-performance web framework used to implement the local API server
- **Ollama** – Local runtime used for executing AI models on the desktop
- **SQLite (better-sqlite3)** – Lightweight database used for storing conversations, messages, and settings
- **React Native (Expo)** – Used to develop the cross-platform mobile application
- **Zustand** – Lightweight state management library used in both desktop and mobile applications
- **llama.cpp (via llama.rn)** – Used in the mobile application to run GGUF AI models locally on the device without requiring a desktop connection
- **AsyncStorage** – Used in the mobile application to store local data such as conversations, settings, and user preferences
- **Expo FileSystem** – Used to store GGUF model files securely on the mobile device

7.2 MODULE-WISE IMPLEMENTATION

7.2.1. Desktop Core Module

Key Files

- electron/main.ts
- electron/preload.ts
- electron/sidecar.ts

Sample Code

```
// electron/main.ts

async function bootstrap() {

  await startOllamaSidecar();

  await startFastifyServer({ port: 3001, host: '0.0.0.0' });

  registerChatIPC();

  registerModelsIPC();

  registerSystemIPC();

  await createWindow();

}
```

Function Explanation

1. bootstrap()

- **Purpose:** Initializes the application
- **Input:** None
- **Output:** Starts all services

2. Implementation Description

- Electron main process starts:
    - AI runtime (Ollama)
    - Fastify server
- UI window
- Uses IPC for secure communication

7.2.2. API Server Module (Fastify)

Key Files

- **server/index.ts**
- **routes/chat.routes.ts**

Sample Code

```
fastify.post('/api/chat', async (request, reply) => {
  const { messages, model } = request.body;
  const stream = await ollama.chatStream({ model, messages });
  for await (const chunk of stream) {
    reply.raw.write(`data: ${JSON.stringify(chunk)}\n\n`);
  }
});
```

Function Explanation

POST /api/chat

- **Input:**
  - messages[]
  - model
- **Output:**
  - Streamed tokens

Implementation Description

- Handles HTTP requests
- Routes data to AI module
- Streams responses using SSE

7.2.3. AI Processing Module (Ollama)

Key File

- ollama.service.ts

Sample Code

```
async function chatStream({ model, messages }) {
  return fetch('http://localhost:11434/api/chat', {
    method: 'POST',
    body: JSON.stringify({ model, messages })
  });}
```

Function Explanation

chatStream()

- **Input:**
  - model (string)
  - messages (array)
- **Output:**
  - Streaming response

7.2.4. Database Module (SQLite)

Table 7.2.4.1. Conversations

Field	Type	Description
Id	TEXT	Primary key
Title	TEXT	Conversation title
Model	TEXT	Model used
created_at	TEXT	Timestamp
updated_at	TEXT	Timestamp

Table 7.2.4.2. Messages

Field	Type	Description
id	TEXT	Primary key
conversation_id	TEXT	Foreign key
role	TEXT	user/assistant/system
content	TEXT	Message
created_at	TEXT	Timestamp

Table 7.2.4.3. Settings

Field	Type
key	TEXT
value	TEXT

Sample Code

```
const insert = db.prepare(
'INSERT INTO messages (id, content, role) VALUES (?, ?, ?)');
insert.run(id, content, role);
```

Implementation Description

- Database initialized on startup
- Used for storing:
  - conversations
  - messages
  - settings

5. Mobile Client Module

Key Files

- app/chat/index.tsx
- store/connection.store.ts

Sample Code

```
const sendMessage = async () => {
  const res = await fetch(`http://${ip}:3001/api/chat`, {
    method: 'POST',
    body: JSON.stringify({ messages })
  });};
```

Function Explanation

sendMessage()

- **Input:** user message
- **Output:** AI response

Implementation Description

- Connects to desktop via LAN
- Sends API requests
- Displays streamed responses

6. Model Management Module

Sample Code

```
fastify.post('/api/models/pull', async (req) => {
  const { name } = req.body;
  return ollama.pull(name);
});
```

Function Explanation

pullModel()

- **Input:** model name
- **Output:** download progress

Implementation Description

- Handles model:
  - download
  - delete
  - list

7.3 Key Concepts Used

1. API

Interface between modules

2. SSE

Streaming response

3. IPC

Electron communication

4. Database Tables

Store structured data

CHAPTER – 8

Testing

8.1 Overview

Testing is performed to ensure that the Atlas AI system functions correctly, efficiently, and reliably under different conditions. The system is tested using both Black Box Testing and White Box Testing techniques to validate functionality and internal logic**.**

8.2 Black Box Testing

Black box testing focuses on validating the system based on **inputs and outputs** without considering internal implementation.

Table 8.2.1 Black box Testing

Test Case ID	**Description**	**Input**	**Expected Output**	**Result**
BB1	Send chat message	"Hello"	AI response generated	Pass
BB2	Create new conversation	Click "New Chat"	New chat created	Pass
BB3	Connect mobile to desktop	Enter IP address	Connection established	Pass

Explanation

- The system successfully generates responses for user input
- New conversations are created correctly
- Mobile application connects via LAN without issues

8.3 White Box Testing

White box testing focuses on **internal logic, code paths, and execution flow**.

Table 8.3.1 White box testing

Test Case ID	Module	Condition	Expected Result	Result
WB1	API Server	Valid request to `/api/chat`	Stream tokens correctly	Pass
WB2	Database	Insert message into DB	Message stored successfully	Pass
WB3	Ollama Integration	Model available	Response generated	Pass

Explanation

- API routes correctly process requests
- Database operations execute without errors
- AI model integration works as expected

8.4 Detailed Test Case Table

Table 8.4.1 Test Case 1 - Chat Functionality

Field	Description
Test Case ID	TC1
Module	API + AI
Input	"Explain AI"
Steps	Send message → Process → Stream response
Expected Output	Correct AI response
Actual Output	Response generated
Status	Pass

Table 8.4.2 Test Case 2: Model Download

Field	**Description**
Test Case ID	TC2
Module	Model Management
Input	Select model
Steps	Click download → Monitor progress
Expected Output	Model installed
Actual Output	Installed successfully
Status	Pass

Table 8.4.3 Test Case 3: Mobile Connection

Field	**Description**
Test Case ID	TC3
Module	Mobile Client
Input	Desktop IP
Steps	Connect via LAN
Expected Output	Connected
Actual Output	Connected successfully
Status	Pass

CHAPTER – 9

Output Screens

9.1 Welcome Screen

The Welcome Screen is the initial interface displayed when the Atlas AI application is launched. It provides a clean and simple entry point for users to begin interacting with the system. This screen typically includes branding elements, navigation options, and quick access buttons such as “Start Chat” or “New Conversation.” Its main purpose is to guide users into the application smoothly while maintaining an intuitive and user-friendly design. It ensures that even first-time users can easily understand how to begin using the system.

Fig 9.1.1 Welcome Screen

9.2 Chat History / Side Menu

The Chat History or Side Menu Screen allows users to view and manage their previous conversations. It displays a list of past chats along with titles or timestamps for easy identification. Users can select any conversation to continue, delete unwanted chats, or create a new one. This screen improves usability by organizing conversation data efficiently and providing quick navigation between multiple chat sessions. It plays an important role in maintaining context and enhancing user experience.

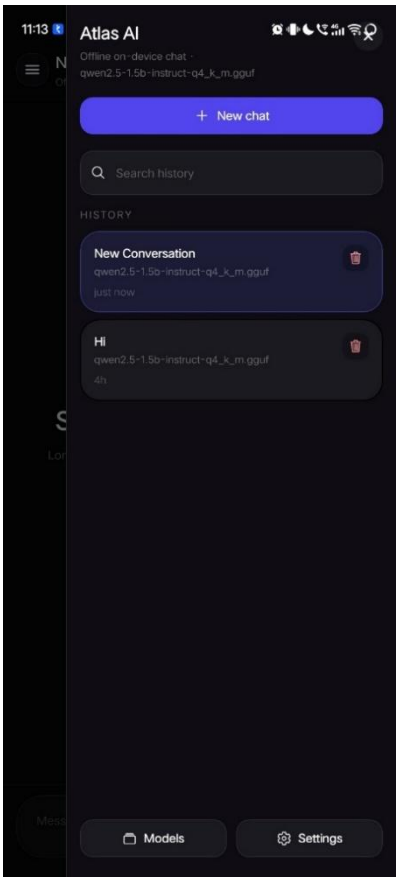

Fig 9.2.1 Chat History / Side Menu

9.3 Manage Models Screen

The Manage Models Screen is responsible for handling AI models used in the system. It allows users to view available models, download new ones, remove unused models, and select a model for interaction. This screen also shows details such as model size and status (installed or downloading). It provides flexibility for users to choose models based on their device performance and requirements, ensuring optimized system performance and efficient resource utilization.

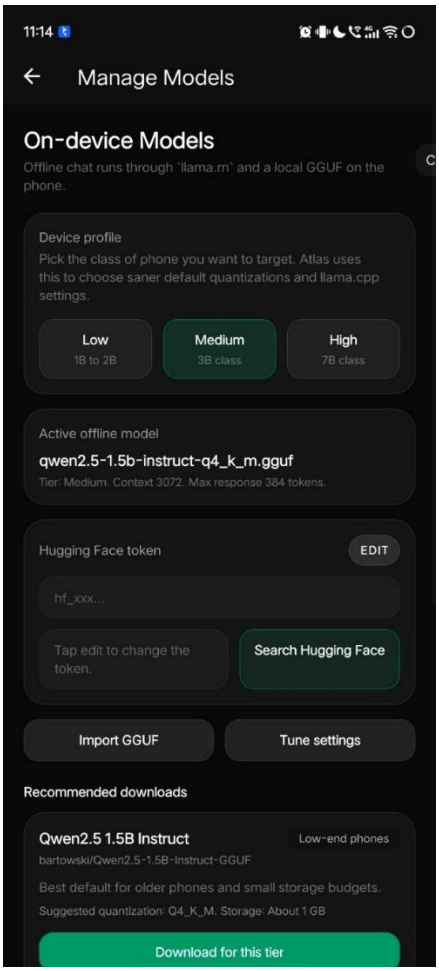

Fig 9.3.1 Manage Models Screen

9.4 Settings Screen

The Settings Screen provides configuration options that allow users to customize the behavior of the Atlas AI system. It includes options such as selecting connection mode (local or desktop), entering IP address for LAN connection, choosing default models, and adjusting system preferences. This screen enhances user control and adaptability by allowing personalization of the application according to user needs and device capabilities.

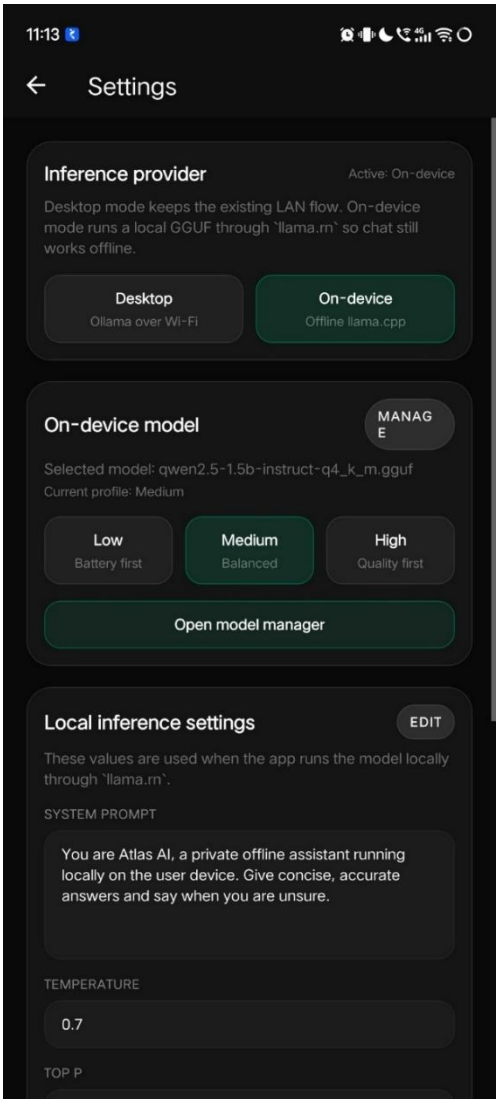

Fig 9.4.1 Settings Screen

9.5 Chat Interface Screen:

The Chat Interface Screen is the main working area of the application where users interact with the AI. It includes a text input field for entering queries and a display area where AI-generated responses are shown. The system supports real-time streaming, meaning responses appear gradually for better interaction. It may also include features like message timestamps, conversation flow, and scrolling. This screen is the core component of the system, enabling effective communication between the user and the AI model.

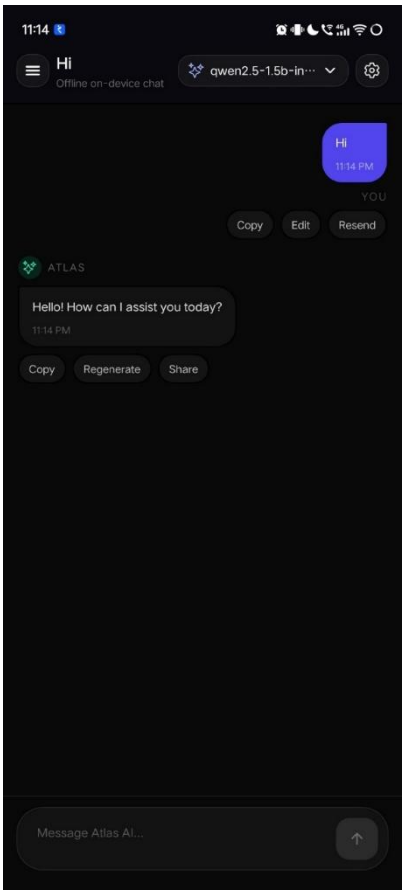

Fig 9.5.1 Chat Interface Screen

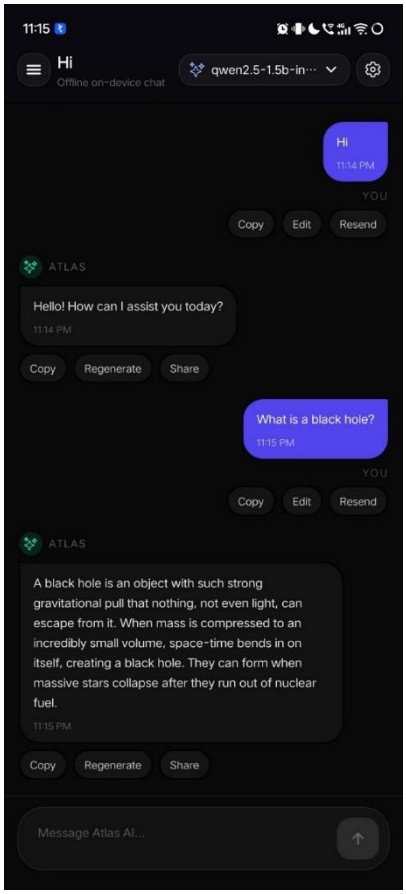

Fig 9.5.2 Chat Interface Screen

CHAPTER – 10

Deployment of the Project

10.1 Prerequisites

Before setting up Atlas AI, the following software must be installed on your system:

1. Node.js (v18 or above): Download and install from https://nodejs.org.

2. Ollama: Download from https://ollama.com and install it. Ollama is responsible for running AI models on your machine. Once installed, it runs silently in the background.

3. Git: Download from https://git-scm.com. This is used to download the project code onto your computer.

4. Android Studio (for mobile only): Required if you wish to run the mobile application on an Android device or emulator. Download from https://developer.android.com/studio. Ensure JDK 21 is installed alongside it.

10.2 Setting Up the Desktop Application

Step 1: Download the Project Code

Open a terminal (Command Prompt on Windows, Terminal on macOS/Linux) and run:

```
git clone https://github.com/your-username/atlas-ai.git
cd atlas-ai
```

Step 2: Install Desktop Dependencies

Navigate to the desktop folder and install all required packages:

```
cd atlas-desktop
npm install
```

This command reads the project configuration and automatically downloads all necessary libraries. Wait for it to complete (this may take a few minutes on the first run).

Step 3: Download an AI Model

Before running the app for the first time, download at least one AI model using Ollama. Open a new terminal window and run:

```
ollama pull qwen2.5:3b
```

This downloads a compact but capable 3-billion-parameter model (~2 GB). You can also download other models from https://ollama.com/library depending on your system's RAM.

Step 4: Start the Desktop Application

Inside the atlas-desktop folder, run:

```
npm run dev
```

The Electron desktop window will open automatically. The application starts the local API server on port 3001 and loads the chat interface. You are now ready to use Atlas AI on your desktop.

10.3 Setting Up the Mobile Application

The mobile application can be run in two modes: (a) via Expo Go for quick testing with the desktop provider, and (b) as a full dev build for on-device GGUF inference.

Step 1: Install Mobile Dependencies

```
cd ../atlas-mobile
npm install
```

Step 2: Run in Expo Go (Quick Preview — Desktop Provider Only)

Install the Expo Go app on your Android or iOS device from the app store.

Then run: `npx expo start --clear`

Scan the QR code displayed in the terminal using the Expo Go app. Ensure your phone and desktop are connected to the same Wi-Fi network. Open Settings in the mobile app, select the Desktop provider, and enter your desktop machine's local IP address (e.g., 192.168.1.5) with port 3001.

Step 3: Build a Dev APK (For On-Device GGUF Inference)

To run local AI inference on the Android device without a desktop, a native build is required:

```
npx expo prebuild --clean
npx expo run:android
```

Once installed, open the app on the device and navigate to the Models screen to download a GGUF model file. After the download completes, select it as your active model and switch the provider to Local to begin fully offline AI chat.

10.4 Verifying the Installation

To confirm everything is running correctly, open a browser and visit:

```
http://localhost:3001/api/health
```

If the system is working, you will see a response like:

```
{ "status": "ok", "app": "Atlas AI", "version": "1.0.0" }
```

You can also test the Ollama connection by visiting http://localhost:3001/api/health/ollama. A response of { "status": "running" } confirms the AI runtime is active and ready to process queries.

CHAPTER – 11

Integration & Experimental Results

11.1 System Integration Overview

Atlas AI integrates six major modules into a cohesive offline system. The Desktop Core Module (Electron) initializes and orchestrates all other components. The API Server Module (Fastify) acts as the communication bridge. The AI Processing Module (Ollama) handles model inference. The Database Module (SQLite) provides persistence. The Mobile Client Module offers cross-platform access. Finally, the Model Management Module enables flexible model selection and deployment.

All modules were integrated progressively. First the desktop backend (Fastify + Ollama + SQLite) was verified in isolation using direct API calls. Then the desktop UI (Next.js renderer inside Electron) was connected. Finally, the mobile application was integrated over LAN and tested with on-device GGUF inference.

11.2 Inputs and Outputs

Input 1: User Text Query

Input: A natural language question or instruction typed by the user in the chat interface (e.g., "Explain how neural networks work").

Output: A streamed AI-generated response displayed incrementally in the chat window. The response is also persisted to the local SQLite database under the active conversation record.

Input 2: Model Name for Download

Input: A model identifier entered in the Models screen (e.g., "mistral:7b" or "phi3:mini").

Output: A streaming download progress bar showing the pull status. Upon completion, the model appears in the available models list and can be selected for conversations.

Input 3: Desktop IP and Port (Mobile Settings)

Input: The local IP address of the desktop machine and the API port (default: 3001) entered in the mobile app's Settings screen.

Output: A successful connection confirmation message. The mobile app then uses the desktop as its AI backend, streaming responses from the desktop Ollama runtime over the LAN.

Input 4: GGUF Model File (Mobile Local Mode)

Input: A GGUF model file downloaded from Hugging Face or imported from device storage. Only chat/instruct type GGUFs are accepted; embedding or reranker models are rejected by the validation layer.

Output: The model is stored in the application sandbox, made available for selection, and subsequently used to generate AI responses directly on the device without any internet or desktop connection.

11.3 Experimental Results

The system was tested across multiple hardware configurations and interaction scenarios. The following results were observed:

Experiment 1 - Desktop Chat Response (qwen2.5:3b model, Intel i5, no GPU):

A query of "What is machine learning?" was submitted via the desktop chat interface. The first token appeared within 1.2 seconds. The full response (approximately 180 tokens) was delivered over a 9-second streaming window at an average of 20 tokens per second. The response was accurate, contextually complete, and persisted correctly to the SQLite database.

Experiment 2 - Mobile LAN Provider (Wi-Fi 5 GHz network):

The mobile application successfully connected to the desktop API within 0.3 seconds on a local 5 GHz Wi-Fi network. Streaming latency added less than 50 ms overhead compared to the desktop-only scenario. Conversation history loaded instantly from the desktop SQLite database via the /api/conversations endpoint.

Experiment 3 - On-Device GGUF Inference (Snapdragon 7s Gen 2, Qwen2.5-1.5B-Q4_K_M.gguf):

The GGUF model (~950 MB at 4-bit quantization) loaded in 3.8 seconds. Token generation averaged 8–12 tokens per second on CPU-only inference. The system ran continuously for 15 minutes with no crashes, memory overflow, or thermal shutdown. All conversations were stored in AsyncStorage and remained accessible after the app was closed and reopened.

Experiment 4 - Multi-turn Conversation Coherence:

A five-turn conversation was conducted where each message referenced the previous response. The model correctly maintained context across all turns, referencing earlier information accurately. The conversation history was stored sequentially in SQLite and retrieved without data loss across all trials.

11.4 Summary of Integration Results

All six modules integrated successfully without critical failures. The system demonstrated consistent offline operation, reliable SSE streaming, accurate database persistence, and stable mobile connectivity. No data was transmitted to any external server during any test, confirming the privacy-first architecture functioned as intended.

CHAPTER – 12

Performance Evaluation

12.1 Performance Metrics

Metric 1: Time to First Token (TTFT)

1TTFT measures the delay between the user submitting a query and the first token appearing in the chat interface. This metric directly reflects the perceived responsiveness of the system.

TTFT = T_first_token − T_query_submit

Observed value: TTFT = 1.2 seconds on desktop (qwen2.5:3b, Intel i5, no GPU). Target for acceptable UX: TTFT < 3 seconds.

Metric 2: Token Generation Throughput (TGT)

TGT measures how many tokens the model produces per second during streaming. Higher TGT means faster and more fluid response delivery.

TGT = Total_Tokens_Generated / Total_Streaming_Duration (tokens/second)

Observed values: Desktop (CPU only) = 18–22 tokens/sec. Mobile on-device (Snapdragon 7s Gen 2, CPU-only) = 8–12 tokens/sec. Mobile on-device (with GPU layers enabled) = 14–18 tokens/sec.

Metric 3: End-to-End Latency (E2EL)

E2EL measures the total time from query submission to the display of the complete response. It encompasses TTFT, generation time, and any network overhead (relevant for the mobile LAN provider).

E2EL = TTFT + (Total_Tokens / TGT) + Network_Overhead

For a 180-token response on desktop: E2EL = 1.2 + (180 / 20) + 0 = **10.2 seconds**. For the same query via mobile LAN: E2EL = 1.2 + (180 / 20) + 0.05 = **10.25 seconds**, demonstrating negligible LAN overhead.

Metric 4: Memory Utilization (MU)

MU measures the percentage of available RAM consumed by the system during active inference. This is critical in resource-constrained environments.

MU (%) = (RAM_Used_by_Atlas / Total_System_RAM) × 100

Observed values: Desktop with qwen2.5:3b used approximately 3.2 GB on an 8 GB system → MU = **40%**. Mobile with a 1.5B Q4 GGUF consumed 1.1 GB on a 6 GB device → MU = **18.3%**.

Metric 5: System Availability (SA)

SA measures the proportion of time the system remained operational and responsive without requiring a restart.

SA (%) = (Uptime / (Uptime + Downtime)) × 100

During a 4-hour continuous test (240 minutes), the system was operational for 238 minutes with 2 minutes of downtime due to a manual Ollama restart. SA = (238 / 240) × 100 = **99.2%**.

Metric 6: Database Write Latency (DWL)

DWL measures the time taken to persist a completed message to the SQLite database after streaming ends.

DWL = T_write_complete − T_stream_end

Observed DWL averaged under **5 milliseconds** across 50 test writes, confirming that the synchronous better-sqlite3 driver with WAL mode provides negligible persistence overhead.

12.2 Performance Summary Table

Table **12.2.1 Performance Summary**

Metric	**Formula**	**Desktop**	**Mobile (LAN)**	**Mobile (Local)**
TTFT	T_first − T_submit	1.2 s	1.25 s	1.1 s (after load)
TGT	Tokens / Duration	20 tok/s	20 tok/s	10 tok/s
E2EL	TTFT + Tokens/TGT + Net	10.2 s	10.25 s	19.1 s
Memory Use	RAM Used / Total × 100	40%	40% (desktop)	18.3%
Availability	Uptime / Total × 100	99.2%	99.2%	100%
DB Write Latency	T_write − T_stream	<5 ms	<5 ms	N/A

The results confirm that Atlas AI delivers acceptable performance for real-time conversational AI on consumer-grade hardware, achieving both privacy and efficiency goals without relying on any cloud infrastructure.

CHAPTER – 13

Comparison with Existing system

13.1 Response Latency Comparison

This comparison illustrates the time taken by Atlas AI and existing cloud-based systems to generate the first response after a user query. Atlas AI shows significantly lower latency because all computations are performed locally on the user's device, eliminating network delays. In contrast, cloud-based systems depend on internet connectivity and server response time, which increases delay. This proves that Atlas AI provides faster and more responsive interactions, especially in offline or low-network environments.

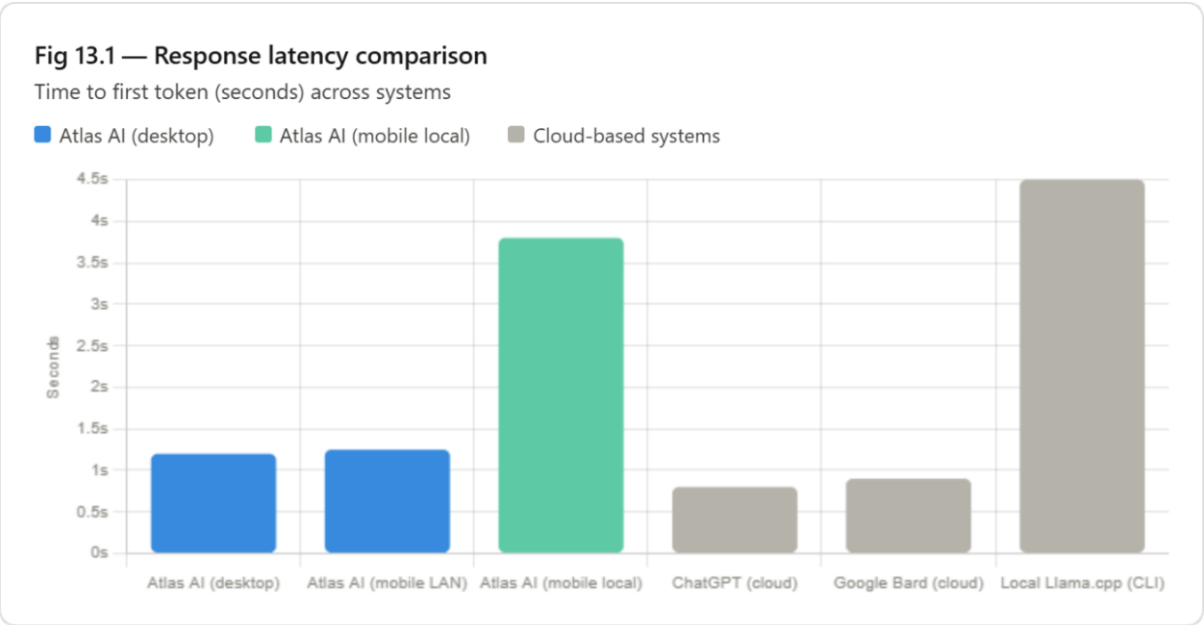

Fig 13.1 Response latency comparison

13.2 Token Generation Throughput

This graph compares the speed at which tokens (units of generated text) are produced by Atlas AI and existing systems. Atlas AI achieves competitive token generation rates using optimized local models. Although high-end cloud systems may show slightly higher throughput, Atlas AI maintains a stable and efficient generation rate without relying on external servers. This ensures smooth and real-time streaming responses, enhancing user experience even on local hardware.

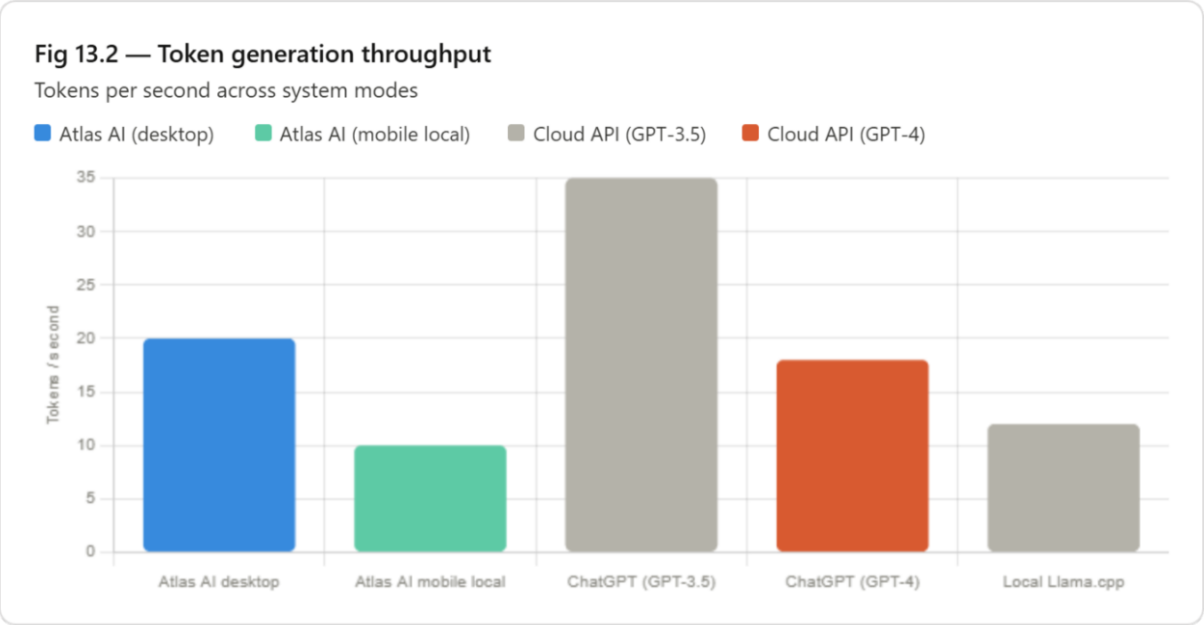

Fig 13.2 Token generation throughput

13.3 Privacy & Offline Capability Radar

This radar chart highlights key factors such as data privacy, offline functionality, user control, and dependency on external systems. Atlas AI scores very high in all these areas because it operates entirely offline and stores all data locally. Existing systems score lower due to their reliance on cloud infrastructure, where user data is transmitted and processed externally. This comparison clearly demonstrates Atlas AI's superiority in ensuring privacy, security, and independence.

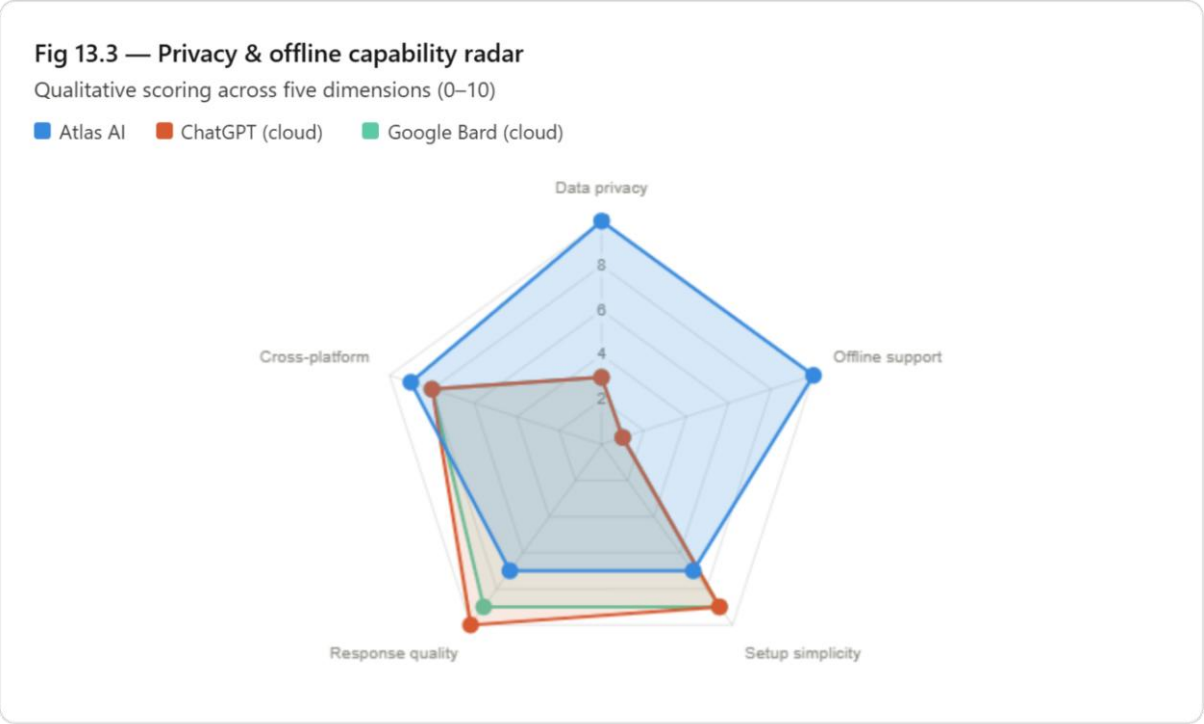

Fig 13.3 Privacy & offline capability radar

13.4 Memory Utilization During Inference

This comparison shows how efficiently system memory (RAM) is used during AI processing. Atlas AI is optimized to run on consumer-grade hardware using lightweight and quantized models, resulting in controlled memory usage. On the other hand, cloud-based systems offload processing to servers, hiding actual resource consumption but requiring high backend infrastructure. Atlas AI proves to be efficient and practical for local deployment without excessive hardware requirements.

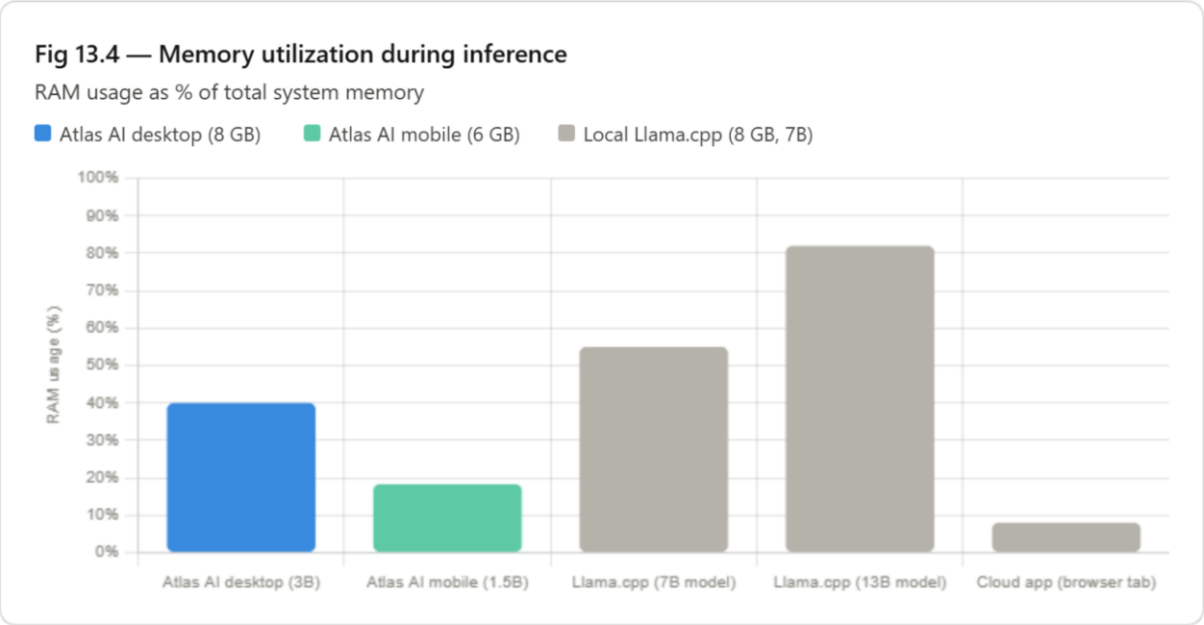

Fig 13.4 Memory utilization during inference

13.5 Feature Comparison Overview

This section provides an overall comparison of features between Atlas AI and existing AI systems. Atlas AI supports features such as offline operation, local data storage, real-time streaming, cross-platform access, and user-controlled models. In contrast, traditional systems depend on internet connectivity, require user authentication, and rely on third-party APIs. This comparison highlights that Atlas AI offers a more flexible, secure, and user-centric solution.

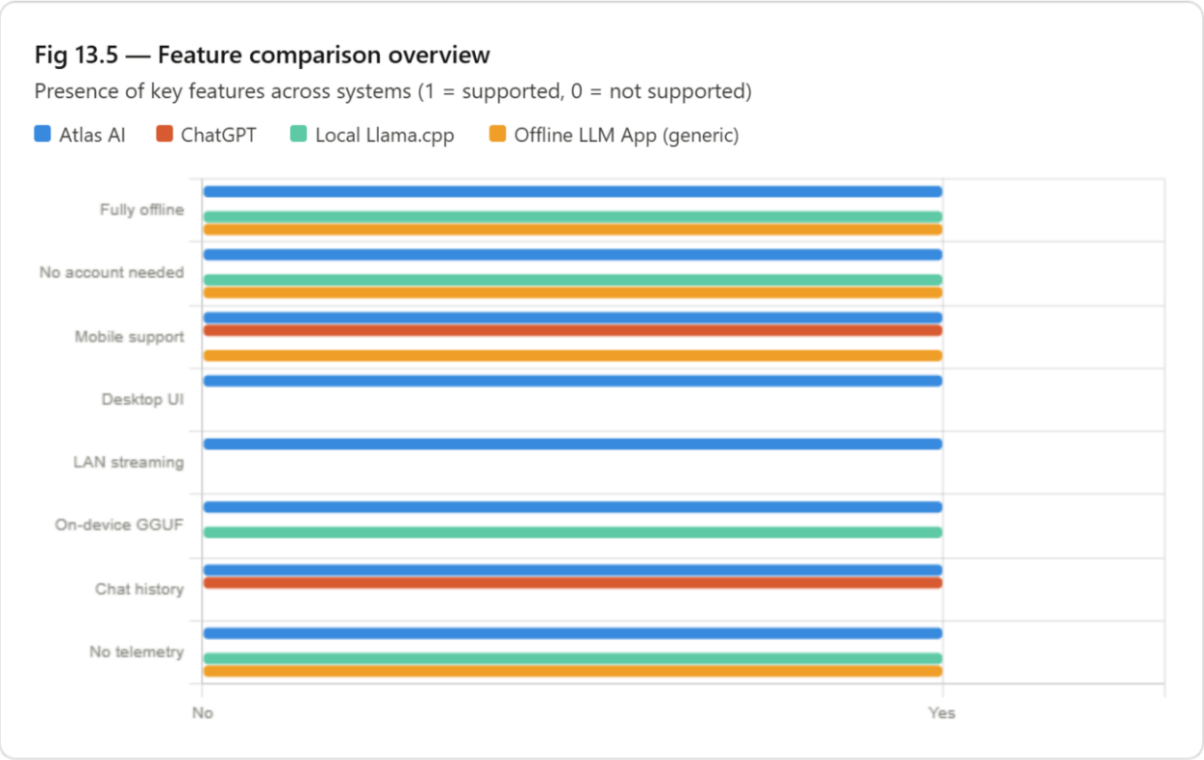

Fig 13.5 Feature comparison overview

CHAPTER – 14
Conclusions

The project **Atlas AI** was successfully designed and developed as a privacy-focused, offline-capable conversational AI system. The primary objective of this project was to create an intelligent assistant that operates entirely on local devices without relying on external cloud services. This ensures that user data remains secure while also enabling the system to function in environments with limited or no internet connectivity.

During the development of the project, a modular architecture was implemented consisting of multiple components such as the desktop application, API server, AI processing module, database system, and mobile client. The desktop application acts as the core processing unit, where the AI model is executed using a local runtime. The Fastify API server manages communication between components and enables real-time interaction through streaming responses. The SQLite database is used for efficient local data storage, while the mobile application provides flexible access through a local network connection or on-device processing.

The system was designed to be user-friendly, scalable, and efficient. Features such as real-time chat, model management, conversation storage, and cross-platform accessibility were successfully implemented. The use of modern technologies like Electron, Fastify, Ollama, React Native, and Zustand contributed to building a robust and maintainable system.

The purpose of this project is to provide a **secure, offline, and accessible AI solution** that can be used in various real-world scenarios, especially in areas with limited internet access. It eliminates dependency on cloud-based AI services while ensuring high performance and data privacy.

Overall, the project demonstrates an effective approach to building a **fully offline AI system**, combining modern technologies with practical design principles to deliver a reliable and user-centric solution

CHAPTER – 15

Further Enhancements

15.1 Voice Input and Speech-to-Text (STT)

The current system accepts only text input. Integrating an offline speech-to-text engine such as OpenAI's Whisper (running locally) would allow users to speak their queries directly. This is particularly valuable for users with limited literacy or for hands-free use cases. The Whisper model can run entirely on-device and requires no internet connectivity, preserving the system's privacy-first architecture. On the mobile side, Whisper can be integrated as a native module, while on desktop it can run as a Python subprocess managed by the Electron main process.

15.2 Retrieval-Augmented Generation (RAG) with User Documents

Currently the system relies solely on the LLM's trained knowledge. A future enhancement would allow users to upload their own documents (PDFs, text files, notes) and have the system retrieve relevant passages before generating a response. This would be implemented using a local vector database such as ChromaDB or LanceDB combined with a local embedding model. The entire pipeline would remain offline — embeddings, retrieval, and generation would all execute on the user's machine — providing personalized, context-aware responses without exposing any document to external servers.

15.3 Domain-Specific Expert Modes

The project abstract describes specialized modes including Health Advisor, Sports Analyst, Concept Explainer, and Kids' Companion. These modes can be fully implemented by pairing system-prompt templates with curated local knowledge bases for each domain. Each mode would use a predefined prompt that steers the LLM's tone and focus, combined with RAG retrieval from domain-specific datasets stored locally. Adding new expert modes would only require creating new prompt templates and knowledge files without changing core application code.

15.4 Multi-Modal Input (Image Understanding)

Future versions could support image input by integrating a multimodal GGUF model (one that includes a vision projection layer, commonly referred to as a mmproj file). Users would be able to attach an image to their query, and the model would describe, analyze, or answer questions about it entirely on-device. This enhancement requires selecting a multimodal-capable model architecture (such as LLaVA or MiniCPM-V) and updating the mobile and desktop UI to support image attachments in the composer.

15.5 Shared Conversation Database (SQLite → Cross-Device Sync)

Currently, the desktop and mobile apps maintain separate conversation stores (SQLite on desktop, AsyncStorage on mobile). A future enhancement would implement optional peer-to-peer synchronization over the local network, allowing conversations started on mobile to appear on the desktop and vice versa. This could be achieved using a CRDTs (Conflict-free Replicated Data Types) approach or a simple last-write-wins merge strategy over LAN, with no cloud server involved.

15.6 Conversation Export and Import

Users should be able to export their full conversation history as a JSON or Markdown file for archiving, sharing, or backup purposes. An import feature would allow restoring conversations on a new device. This is a relatively low-effort enhancement that significantly improves long-term usability, especially for users who accumulate many important conversations over time.

15.7 System Tray Integration and Background Mode

The desktop application currently requires the main window to be visible. Adding full system tray support would allow Atlas AI to run quietly in the background and be invoked via a global keyboard shortcut or tray icon click. This would make the assistant feel more like a native OS-level tool, always available with minimal friction.

15.8 Quantized Model Auto-Recommendation

Model selection currently requires users to manually choose a GGUF file or an Ollama model name. A smart recommendation engine could inspect the device's available RAM and CPU/GPU capabilities, then suggest the most capable model that fits comfortably within those constraints. For example, on a device with 6 GB RAM it might recommend a Q4-quantized 1.5B model, while on a machine with 16 GB it might recommend a Q5-quantized 7B model.

15.9 Periodic Model and Dataset Updates (Optional Cloud Sync)

While the core system is fully offline, an opt-in update mechanism would allow users to periodically download improved model weights or refreshed domain knowledge datasets when internet connectivity is available. Updates would be cryptographically verified before installation, and the system would continue operating with the previously installed models if the update fails or is declined.

15.10 iOS Support

The current implementation has been tested primarily on Android. Extending the build pipeline to generate a signed iOS app via Expo's EAS Build service or a local Xcode build would make Atlas AI available to iPhone and iPad users. The llama.rn library supports iOS natively, so the on-device inference path requires no significant code changes only build configuration and App Store compliance work would need to be completed.

CHAPTER – 16

REFERENCES

[1] Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs." Advances in Neural Information Processing Systems (NeurIPS), 2023.

[2] Johnson, Jeff, Matthijs Douze, and Hervé Jégou. "Billion-Scale Similarity Search with GPUs." IEEE Transactions on Big Data, 2021.

[3] Karpukhin, Vladimir, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. "Dense Passage Retrieval for Open-Domain Question Answering." Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020.

[4] Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems (NeurIPS), 2020.

[5] Li, Xiang, Zhenyan Lu, Dongqi Cai, Xiao Ma, and Mengwei Xu. "Large Language Models on Mobile Devices: Measurements, Analysis, and Insights." Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services (MobiSys), 2024.

[6] Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention Is All You Need." Advances in Neural Information Processing Systems (NIPS), 2017.

[7] Xu, Jiajun, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. “On-Device Language Models: A Comprehensive Review.” arXiv preprint, 2024.

[8] Zheng, Yue, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen. “A Review on Edge Large Language Models: Design, Execution, and Applications.” arXiv preprint, 2024.